

Bausteine Computergestützter Datenanalyse

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Franca Hollmann
Maik Poetzsch	Sebastian Seipel	

2026-09-16

Table of contents

Tool Module Pandas	4
Prerequisites	4
Learning Objectives	5
1 Introduction	6
1.1 The Series and DataFrame Data Structures	6
1.1.1 Series	7
1.1.2 Series Exercise	9
1.1.3 DataFrame	9
1.1.4 DataFrame Exercise	14
1.2 Descriptive Data Analysis with Pandas	15
1.3 Slicing	19
1.3.1 Slice Operator	19
1.3.2 Slicing with Pandas Methods	20
1.3.3 Index-based Slicing with .iloc[]	22
1.4 Slicing Exercises	24
1.5 Combining Data Structures	26
1.6 Inserting and Deleting in Data Structures	28
1.7 Exercises: Combining and Deleting	28
References	30
2 Operations	31
2.1 Row- and Column-wise Operations	32
2.1.1 Arithmetic Functions	33
2.1.2 Summary Functions	34
2.1.3 Boolean Functions	35
2.2 Single Values or List	35
2.3 NumPy Array	36
2.4 Series	37
2.5 DataFrame	38
2.5.1 Using the .agg() and .apply() Methods	40
2.6 Function	40
2.7 Function Name	41
2.8 List of Functions	42
2.9 Dictionary of Functions	42

2.10	Exercises: Operations	43
2.11	Search and Replace	45
2.12	Exercises: Search and Replace	46
2.13	Sorting	47
2.14	Exercises: Sorting	50
2.15	GroupBy	51
2.16	DataFrame guinea_pigs	51
2.17	guinea_pigs grouped by Supplement Type	52
2.18	Tooth Length by Supplement Type	52
2.19	Tooth Length by Supplement Type and Dose	52
2.20	Exercises: GroupBy	53
3	Plot Creation	56
3.1	Series	56
3.2	DataFrame	58
3.3	Subplots	60
4	Data Types	64
5	Time Series	67
5.1	Date and Time Information in Python	67
5.1.1	Naive and Aware Datetime Objects	68
5.1.2	Everything is Relative: The Epoch	84
5.1.3	Daylight Saving Time	85
5.1.4	Calendar	86
5.2	datetime in Pandas	86
5.3	timedelta in Pandas	89
5.4	Accessing Time Series	90
5.5	Exercises	93
6	Reading and Writing Files	95
6.1	Reading Time Series	100
6.2	Time Series Reading Exercises	102
6.3	Reading Complex Files	104

Tool Module Pandas



Building Blocks of Computer-Assisted Data Analysis by Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, and Sebastian Seipel. Tool Module Pandas by Marc Fehr and Maik Poetzsch is licensed under [CC BY 4.0](#). The work is available on [GitHub](#). Excluded from the license are all third-party logos and other content marked otherwise. 2025

Suggested Citation

Arnold, Lukas, Simone Arnold, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, and Sebastian Seipel. 2025. “Building Blocks of Computer-Assisted Data Analysis. Tool Module Pandas”. <https://github.com/bausteine-der-datenanalyse/w-pandas-en>.

BibTeX Template

```
@misc{BCD-w-pandas-2025,  
  title={Building Blocks of Computer-Assisted Data Analysis. Tool Module Pandas},  
  author={Arnold, Lukas and Arnold, Simone and Baitsch, Matthias and Fehr, Marc and Hollmann, Franca and Poetzsch, Maik and Seipel, Sebastian},  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/w-pandas-en}}
```

Prerequisites

The following modules should already have been completed:

- w-Python-en
- w-NumPy-en

This module uses the following libraries:

- NumPy
- Pandas
- Matplotlib
- zoneinfo (optional)

The module works with the following datasets:

- Tooth growth in guinea pigs [CSV file](#)
- Vehicle data from *Motor Trend* magazine [GitHub](#)
- Penguin measurements from Palmer Station [GitHub](#)
- Microsoft stock price data [Kaggle](#)

Learning Objectives

In this module, you will learn ...

- the data structures of the Pandas module: Series and DataFrame.
- how operations are executed in Pandas.
- how to create plots using Pandas.
- how to read (and write) files.

1 Introduction

The Pandas module was designed for working with structured data. Pandas makes analysis particularly easy for data presented in tabular form, as it provides the `DataFrame`—a user-friendly structure for handling different data types and missing values. Like NumPy, Pandas allows vectorized operations without having to loop through each element of a collection. Additionally, Pandas integrates functionalities from other modules and offers a unified interface for:

- Date and time information
- Plotting
- Reading files

The Pandas module is loaded using the command `import pandas`. The abbreviation `pd` is commonly used. Since Pandas is built on top of NumPy, both modules are often imported together. Many functions and methods from NumPy and Pandas are compatible with each other.

```
import numpy as np
import pandas as pd
```

1.1 The Series and DataFrame Data Structures

Pandas introduces two classes: `Series` and `DataFrame`.

- `Series` are one-dimensional arrays that contain exactly one data type.
- `DataFrame` are two-dimensional arrays, composed column-wise of `Series`, allowing them to hold different data types. (Through hierarchical indexing, multi-dimensional data structures are possible, see [MultiIndex](#).)

Both data structures have an index, which is displayed in the output.

The index starts at 0, just like in base Python.

```
0    Morning shift
1    Morning shift
```

```
2    Evening shift
dtype: string
```

The index is numeric by default but can be set to any values.

The index can be customized.

```
Monday      Morning shift
Tuesday     Morning shift
Wednesday   Evening shift
dtype: string
```

1.1.1 Series

Series are created using the function `pd.Series(data)`. The data can be a single value, a collection, or a NumPy array.

```
single_value_series = pd.Series('Hello World!')
print(f"Series from a single value:\n{single_value_series}")

numeric_series = pd.Series([1, 2, 3])
print(f"\nSeries from a list:\n{numeric_series}")

alphanumeric_series = pd.Series(('a', '5', 'g'))
print(f"\nSeries from a tuple:\n{alphanumeric_series}")

boolean_series = pd.Series(np.array([True, False, True])) # NumPy array
print(f"\nSeries from a NumPy array:\n{boolean_series}")
```

```
Series from a single value:
0    Hello World!
dtype: str
```

```
Series from a list:
0    1
1    2
2    3
dtype: int64
```

```
Series from a tuple:
0    a
1    5
```

```
2    g
dtype: str
```

Series from a NumPy array:

```
0    True
1   False
2    True
dtype: bool
```

When creating a `pd.Series`, various parameters can be passed:

- `pd.Series(data, dtype='float')` sets the data type of the Series.
- `pd.Series(data, index=['A1', 'B2', 'C3'])` provides values for the index.
- `pd.Series(data, name='the name')` assigns a name to the Series.

```
numeric_series = pd.Series([1, 2, 3], dtype='float', index=['A1', 'B2', 'C3'], name='Floating point numbers')
print(numeric_series)
```

```
A1    1.0
B2    2.0
C3    3.0
Name: Floating point numbers, dtype: float64
```

For an existing Series, the name and index can be accessed and modified via the corresponding attributes. To change the data type, the method `pd.Series.astype()` is used. An overview of the data types available in Pandas can be found in the [Pandas documentation](#).

```
print(f"Name of the Series: {numeric_series.name}")
numeric_series.name = 'Floating point numbers'

print(f"Index of the Series: {numeric_series.index}")
numeric_series.index = ['one', 'two', 'three']

numeric_series = numeric_series.astype('string')
print(f"\nThe modified Series:\n{numeric_series}")
```

```
Name of the Series: Floating point numbers
Index of the Series: Index(['A1', 'B2', 'C3'], dtype='str')
```

The modified Series:

```
one      1.0
two      2.0
three    3.0
Name: Floating point numbers, dtype: string
```

1.1.2 Series Exercise

Change the data type of the object `numeric_series` to integer and choose a new name for the Series.

💡 Tip 1: Example solution dtype

```
numeric_series.name = 'Integers'
numeric_series = numeric_series.astype('float')
numeric_series = numeric_series.astype('int')

print(numeric_series)
```

```
one      1
two      2
three    3
Name: Integers, dtype: int64
```

1.1.3 DataFrame

A `DataFrame` is created using the function `pd.DataFrame([data])`. The `data` is list-like but can consist of a single value, a Series, a NumPy array, or multiple Series and collections.

```
single_value_df = pd.DataFrame(['Hello World!'])
print(single_value_df, "\n")

df_from_lists = pd.DataFrame([[1, 2, 3], [4, 5, 6]])
print(df_from_lists, "\n")

df_from_series = pd.DataFrame([alphanumeric_series, boolean_series])
print(df_from_series, "\n")

df_from_mixed = pd.DataFrame([np.array([True, False, True]), alphanumeric_series, [1, 2, 3]])
print(df_from_mixed)
```

```

0
0 Hello World!

```

```

0 1 2
0 1 2 3
1 4 5 6

```

```

0 1 2
0 a 5 g
1 True False True

```

```

0 1 2
0 True False True
1 a 5 g
2 1 2 3

```

When creating a `DataFrame`, various parameters can also be passed:

- `pd.DataFrame(data, dtype='float')` sets the data type for all values in the `DataFrame`. If this parameter is not provided, Pandas selects an appropriate data type for each column.
- `pd.DataFrame(data, index=['A1', 'B2', 'C3'])` provides values for the index.
- `pd.DataFrame(data, columns=['Column1', 'Column2'])` provides values for the column labels.

To enter data column-wise, the `DataFrame` can be transposed using the `DataFrame.T` attribute. In this case, the column labels must be passed as the `index` argument and the index labels as the `columns` argument.

```

df_transposed = pd.DataFrame([[1, 2, 3], [True, False, True]], index=['Column 1', 'Column 2'],
print(df_transposed)

```

```

Column 1 Column 2
Row 1      1      True
Row 2      2     False
Row 3      3      True

```

A direct assignment of labels is possible by first creating the transposed `DataFrame` and then setting the labels using the `.index` and `.columns` attributes.

```
df_transposed = pd.DataFrame([[1, 2, 3], [True, False, True]]).T
df_transposed.columns = ['Column 1', 'Column 2']
df_transposed.index = ['Row 1', 'Row 2', 'Row 3']
print(df_transposed)
```

	Column 1	Column 2
Row 1	1	True
Row 2	2	False
Row 3	3	True

Creating transposed DataFrames has the drawback that Pandas manages the data types column-wise. When data of different types is entered row-wise, a data type suitable for all columns is chosen. In the following example, Pandas therefore selects the `object` data type for mixed types.

```
df_transposed = pd.DataFrame([[1, 2, 3], ['a', 'b', 'c']], index=['Numbers', 'Letters']).T
print(df_transposed)
print(f"\n{df_transposed.dtypes}")
```

	Numbers	Letters
0	1	a
1	2	b
2	3	c

```
Numbers    object
Letters    object
dtype: object
```

Alternatively, a DataFrame can be created directly from a dictionary. In this case, the keys are used as column labels.

```
df = pd.DataFrame({'Column 1': [1, 2, 3], 'Column 2': [4.1, 5.6, 6.0]}, index=['top', 'middle', 'bottom'])
print(df)
```

	Column 1	Column 2
top	1	4.1
middle	2	5.6
bottom	3	6.0

Additionally, a DataFrame can be expanded by assigning data.

```
# create an empty DataFrame
df = pd.DataFrame()

# assign data
df['Column label'] = [1, 2, 3]
df['Second column'] = alphanumeric_series

print(df)
```

	Column label	Second column
0	1	a
1	2	5
2	3	g

Tip 2: The Index

In most cases, an index ranging from 0 to n-1 is the most practical. A numeric index is helpful for selecting index ranges (slicing) and working with multiple data structures. Try creating a DataFrame from two Series with different indices to see what happens.

Also, placing descriptive or measured variables in the index contradicts the concept of tidy data, a system for structuring datasets introduced in the [Structured Data Import Method Module](#).

Existing DataFrames can be modified similarly to Series. To change the data type of one or more columns, use the method `pd.DataFrame.astype()`.

```
df = pd.DataFrame({'Column 1': ['1', '2', '3'], 'Column 2': [True, False, True]})
print(f"The data types of df:\n{df.dtypes}")

# change data type of Column 1
df['Column 1'] = df['Column 1'].astype('string')
print(f"\nThe data types of df:\n{df.dtypes}")
```

The data types of df:

Column 1	str
Column 2	bool
dtype:	object

The data types of df:

```
Column 1    string
Column 2      bool
dtype: object
```

Similarly, a data type can be assigned to all columns of a DataFrame.

```
df = df.astype('string')
print(f"\nThe data types of df:\n{df.dtypes}")
```

```
The data types of df:
Column 1    string
Column 2    string
dtype: object
```

To assign different data types, a dictionary is used.

```
df = df.astype({'Column 1': 'int', 'Column 2': 'bool'})
print(f"\nThe data types of df:\n{df.dtypes}")
```

```
The data types of df:
Column 1    int64
Column 2      bool
dtype: object
```

The column names and index of an existing **DataFrame** can be modified using the corresponding attributes or methods. Column names can be changed via the `pd.DataFrame.columns` attribute by passing a list whose length matches the number of columns. The index can be modified via the `pd.DataFrame.index` attribute by assigning a list whose length matches the number of rows.

```
# change column names using the .columns attribute
df.columns = ['Column1', 'Column2']
df.index = [1, 2, 3]
print(df)
```

```
   Column1  Column2
1         1      True
2         2      True
3         3      True
```

With the method `pd.DataFrame.rename(columns={"old1": "new1", "old2": "new2"}, index={"old1": "new1", "old2": "new2"}, inplace=True)`, column and row labels can be passed as a dictionary. This allows changing all or selected labels. Using the argument `inplace=True` applies the changes directly without reassigning the object.

```
df.rename(columns={'Column1': 'Column_1', 'Column2': 'Column_2'}, index={1: 'A1', 2: 'B2', 3: 'C3'}, inplace=True)
print(df)
```

	Column_1	Column_2
A1	1	True
B2	2	True
C3	3	True

The method `pd.DataFrame.reset_index(inplace=True, drop=True)` resets the index to the default values. If the parameter `drop=False` is set, the old index is added as a column at index position 0 in the DataFrame.

```
df.reset_index(inplace=True, drop=True)
print(df)
```

	Column_1	Column_2
0	1	True
1	2	True
2	3	True

1.1.4 DataFrame Exercise

Create a DataFrame.

- The first column should contain the numbers from 1 to 12 and be labeled 'Number'. The second column should contain the month names of the year and be labeled 'Month'.
- Afterwards, add the Series `holidays` as a third column with the label 'Holidays'.
`holidays = [False, False, False, True, False, True, True, True, False, True, False, True]`

💡 Tip 3: Example Solution

```
holidays = [False, False, False, True, False, True, True, True, False, True, False, True]

df = pd.DataFrame({
    'Number': list(range(1, 13)),
    'Month': ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December'],
})

df['Holidays'] = holidays

print(df)
```

	Number	Month	Holidays
0	1	January	False
1	2	February	False
2	3	March	False
3	4	April	True
4	5	May	False
5	6	June	True
6	7	July	True
7	8	August	True
8	9	September	False
9	10	October	True
10	11	November	False
11	12	December	True

1.2 Descriptive Data Analysis with Pandas

Pandas provides several convenient functions to describe the structure of a dataset and the data it contains. As an example dataset, we use data on the length of odontogenic cells in guinea pigs that received Vitamin C either directly (VC) or in the form of orange juice (OJ) at different doses.

Code Block 1.1

```
filepath = "01-daten/ToothGrowth.csv"
guinea_pigs = pd.read_csv(filepath_or_buffer=filepath, sep=',', header=0,
    names=['ID', 'len', 'supp', 'dose'], dtype={'ID': 'int', 'len': 'float', 'dose': 'float',
```

Crampton, E. W. 1947. "THE GROWTH OF THE ODONTOBLASTS OF THE INCISOR

TOOTH AS A CRITERION OF THE VITAMIN C INTAKE OF THE GUINEA PIG.” The Journal of Nutrition 33 (5): 491–504. <https://doi.org/10.1093/jn/33.5.491>

The dataset can be accessed in R using the command `ToothGrowth`.

A snippet of the dataset:

	ID	len	supp	dose
0	1	4.2	VC	0.5
10	11	16.5	VC	1.0
20	21	23.6	VC	2.0
30	31	15.2	OJ	0.5
40	41	19.7	OJ	1.0
50	51	25.5	OJ	2.0

The method `pd.DataFrame.info()` generates a summary of the dataset.

```
guinea_pigs.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 60 entries, 0 to 59
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   ID      60 non-null        int64
1   len     60 non-null        float64
2   supp    60 non-null        category
3   dose    60 non-null        float64
dtypes: category(1), float64(2), int64(1)
memory usage: 1.7 KB
```

The dimensions of a Series or a DataFrame can be accessed using the `shape` attribute. The DataFrame has 60 rows and 4 columns.

```
print(guinea_pigs.shape)
```

```
(60, 4)
```

The method `pd.DataFrame.describe()` generates descriptive statistics for a DataFrame. By default, all numeric columns are considered. Using the `include` parameter, you can specify which columns to include. `include='all'` includes all columns, which may not always be meaningful, as it will also summarize the column with the guinea pigs' ID numbers.

```
print(guinea_pigs.describe(include='all'))
```

	ID	len	supp	dose
count	60.000000	60.000000	60	60.000000
unique	NaN	NaN	2	NaN
top	NaN	NaN	OJ	NaN
freq	NaN	NaN	30	NaN
mean	30.500000	18.813333	NaN	1.166667
std	17.464249	7.649315	NaN	0.628872
min	1.000000	4.200000	NaN	0.500000
25%	15.750000	13.075000	NaN	0.500000
50%	30.500000	19.250000	NaN	1.000000
75%	45.250000	25.275000	NaN	2.000000
max	60.000000	33.900000	NaN	2.000000

Using the `include` parameter, you can pass a list of data types to include. The `exclude` parameter works similarly to exclude certain data types from the output.

```
print(guinea_pigs.describe(include=['float']))
```

	len	dose
count	60.000000	60.000000
mean	18.813333	1.166667
std	7.649315	0.628872
min	4.200000	0.500000
25%	13.075000	0.500000
50%	19.250000	1.000000
75%	25.275000	2.000000
max	33.900000	2.000000

```
print(guinea_pigs.describe(include=['category']))
```

	supp
count	60
unique	2
top	OJ
freq	30

The method `pd.DataFrame.count()` counts all non-missing values in each column, or with `pd.DataFrame.count(axis='columns')` in each row.

```
guinea_pigs.count(axis='rows') # the default value of axis is 'rows'
```

```
ID      60
len      60
supp     60
dose     60
dtype: int64
```

The method `pd.Series.value_counts()` counts the occurrences of each unique value in a Series. This method can also be applied to a DataFrame, in which case it counts the frequency of each unique row (which is not meaningful in this context).

```
guinea_pigs['dose'].value_counts()
```

```
dose
0.5    20
1.0    20
2.0    20
Name: count, dtype: int64
```

The method `pd.unique()` lists all unique values in a Series.

```
guinea_pigs['dose'].unique()
```

```
array([0.5, 1. , 2. ])
```

1.3 Slicing

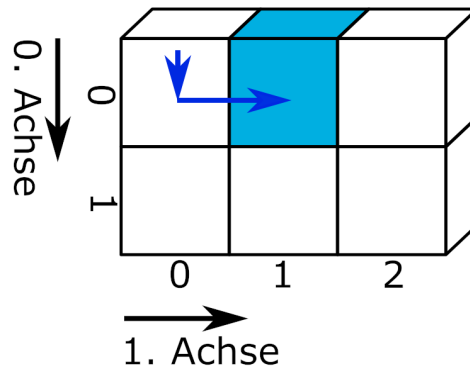


Figure 1.1: two-dimensional dataset

Slicing by Marc Fehr is licensed under [CC-BY-4.0](#) and available on [GitHub](#). The graphic has been cropped to the shown part and the top label removed. 2024

Pandas provides its own tools for selecting index ranges. The slice operator from base Python is therefore only briefly introduced.

1.3.1 Slice Operator

With the slice operator, index ranges can be selected from a Series, similar to a list.

```
ten_numbers = pd.Series(range(0, 10))
print(ten_numbers[3:6])
```

```
3    3
4    4
5    5
dtype: int64
```

The slice operator selects rows from a DataFrame.

```
print(guinea_pigs[7:12])
```

	ID	len	supp	dose
7	8	11.2	VC	0.5
8	9	5.2	VC	0.5
9	10	7.0	VC	0.5
10	11	16.5	VC	1.0
11	12	16.5	VC	1.0

By specifying a column name, the corresponding column is selected and returned as a Series. By appending a second slice operator, it is possible to retrieve values within a specific index range, similar to a one-dimensional dataset. This is called chained indexing.

```
print(guinea_pigs['dose'][10:15], "\n")
print(type(guinea_pigs['dose'][10:15]))
```

```
10    1.0
11    1.0
12    1.0
13    1.0
14    1.0
Name: dose, dtype: float64

<class 'pandas.Series'>
```

⚠ Warning 1: Chained Indexing

Chained indexing in Pandas may return either a copy of the object or a reference to the object's memory, depending on the context. In Pandas 3.0, chained indexing will no longer be supported, and creating a copy will become the standard. More information can be found at the linked resource.

“Whether a copy or a reference is returned for a setting operation may depend on the context. This is sometimes called **chained assignment** and should be avoided. See [Returning a View versus Copy](#).”

([Pandas Documentation](#))

1.3.2 Slicing with Pandas Methods

For slicing Series and DataFrames, Pandas provides the methods `.iloc[]` and `.loc[]`.

- `.loc[]` works with index or column labels, but also accepts a boolean array.

- `.iloc[]` works with integers, but also accepts a boolean array.

For slicing a Series, a range can be passed, e.g., `pd.Series.iloc[5:8]`. For slicing a DataFrame, two comma-separated ranges are passed: the first for rows, the second for columns, e.g., `pd.DataFrame.iloc[5, 2:4]`. To select all rows or columns, a colon can be used, e.g., `pd.DataFrame.iloc[5, :]`.

Label-based Slicing with `.loc[]`

For a Series, `.loc` interprets passed characters as index labels. Letters and other characters are passed as strings in quotes, e.g., `.loc['e']`; numbers without quotes. In addition to single values ('a' or 0), lists or arrays (['a', 'b', 'c'] or [1, 2, 3]) and slices can be passed ('a':'c' or 0:2). Slicing with a single value returns a scalar.

Warning 2: Inclusive Slicing

Unlike base Python and slicing with `.iloc[]`, label-based slicing in Pandas is inclusive, meaning the last selected position is included.

```
# Numbers
ten_numbers = pd.Series(range(0, 10))
print("Return a single value:", ten_numbers.loc[5]) # single value
print(ten_numbers.loc[[2, 4, 7]]) # list
print(ten_numbers.loc[5:7], "\n") # slice

# Letters and other characters
six_numbers = pd.Series(list(range(0, 6)), index=['a', 'b', 'c', 'd', 'e', 'f'])
print("Return a single value:", six_numbers.loc['c']) # single value
print(six_numbers.loc[['c', 'f', 'a']]) # list
print(six_numbers.loc['c':'e']) # slice
```

Return a single value: 5

```
2    2
4    4
7    7
dtype: int64
5    5
6    6
7    7
dtype: int64
```

Return a single value: 2

```
c    2
f    5
a    0
dtype: int64
c    2
d    3
e    4
dtype: int64
```

The interpretation as labels means that with a non-numeric index, passed numbers are not found.

```
try:
    print(six_numbers.loc[2:4])
except Exception as error:
    print(error)
```

cannot do slice indexing on Index with these indexers [2] of type int

For DataFrames, slicing works in the same way.

```
print(guinea_pigs.loc[18:22, ['len', 'dose']])
```

	len	dose
18	18.8	1.0
19	15.5	1.0
20	23.6	2.0
21	18.5	2.0
22	33.9	2.0

1.3.3 Index-based Slicing with `.iloc[]`

The `.iloc[]` method allows selecting slices based on index positions. The method accepts the same types of inputs as `.loc[]`.

Warning 3: Exclusive Slicing

When slicing with the `.iloc[]` method, Pandas behaves like base Python, counting exclusively.

Slicing with single values returns a single value. The method also accepts a list or a slice.

```
print("Return a single value:", guinea_pigs.iloc[27, 2]) # single value
print(guinea_pigs.iloc[[27, 29, 52], 2:4]) # list and slice
```

Return a single value: VC

	supp	dose
27	VC	2.0
29	VC	2.0
52	OJ	2.0

The Methods `.head()` and `.tail()`

Simplified versions of index-based slicing are the methods `.head(n=5)` and `.tail(n=5)`, which return the first or last `n` rows of a DataFrame or Series. The optional parameter `n` controls the number of displayed rows. These methods are useful for quickly getting an overview of a dataset.

```
print(guinea_pigs.head(3), "\n")
print(guinea_pigs.tail(3))
```

	ID	len	supp	dose
0	1	4.2	VC	0.5
1	2	11.5	VC	0.5
2	3	7.3	VC	0.5

	ID	len	supp	dose
57	58	27.3	OJ	2.0
58	59	29.4	OJ	2.0
59	60	23.0	OJ	2.0

Series can also be viewed using these methods.

```
print(guinea_pigs['len'].tail(3))
```

```
57    27.3
58    29.4
59    23.0
Name: len, dtype: float64
```

1.4 Slicing Exercises

1. Given a Pandas Series `temperatures_2021` with the average monthly temperatures, select the temperatures for the spring months (March to May).

```
temperatures_2021 = pd.Series(  
    [2, 4, 7, 12, 19, 23, 25, 23, 18, 15, 9, 5],  
    index=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',  
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']  
)
```

2. Select the temperatures for the last three months of the year once using the slice operator and once using the Pandas methods.
3. Using the `.loc[]` method, select the columns 'dose' and 'len' from the `guinea_pigs` DataFrame and display the first 4 and last 3 rows. (Code to read the file see Code Block 1.1)
4. The methods `.loc[]` and `.iloc[]` also accept a boolean array as input. From the 'dose' column of the `guinea_pigs` DataFrame, return all rows with the value 2.0.

💡 Tip 4: Example Solution Slicing

Exercise 1

```
print(temperatures_2021.loc[['Mar', 'Apr', 'May']])
```

```
Mar      7  
Apr     12  
May     19  
dtype: int64
```

Exercise 2

```
print(temperatures_2021[len(temperatures_2021)-3:], "\n")  
print(temperatures_2021.iloc[len(temperatures_2021)-3:])
```

```
Oct      15  
Nov       9  
Dec       5  
dtype: int64
```

```
Oct      15
```

```
Nov      9
Dec      5
dtype: int64
```

Exercise 3

```
print(guinea_pigs.loc[:, ['dose', 'len']].head(n=4), "\n")
print(guinea_pigs.loc[:, ['dose', 'len']].tail(n=3))
```

```
   dose  len
0   0.5  4.2
1   0.5 11.5
2   0.5  7.3
3   0.5  5.8
```

```
   dose  len
57  2.0 27.3
58  2.0 29.4
59  2.0 23.0
```

Exercise 4

```
# Slice from Series
# print(guinea_pigs['dose'].loc[guinea_pigs['dose'] == 2.0])

# Slice from DataFrame
print(guinea_pigs.loc[guinea_pigs['dose'] == 2.0, ['dose']])
```

```
   dose
20  2.0
21  2.0
22  2.0
23  2.0
24  2.0
25  2.0
26  2.0
27  2.0
28  2.0
29  2.0
50  2.0
51  2.0
52  2.0
```

```

53    2.0
54    2.0
55    2.0
56    2.0
57    2.0
58    2.0
59    2.0

```

1.5 Combining Data Structures

DataFrames are flexible data storage structures. Using the function `pd.concat()`, Series and DataFrames can be merged.

- Using the argument `pd.concat(ignore_index=True)` generates a new index.
- Using the argument `pd.concat(axis=1)` merges the passed objects column-wise.

```

series_1 = pd.Series([1, 2])
series_2 = pd.Series([4, 5])
print(pd.concat([series_1, series_2]), "\n")
print(pd.concat([series_1, series_2], ignore_index=True), "\n")
print(pd.concat([series_1, series_2], ignore_index=True, axis=1))

```

```

0    1
1    2
0    4
1    5
dtype: int64

```

```

0    1
1    2
2    4
3    5
dtype: int64

```

```

   0  1
0  1  4
1  2  5

```

Similarly, DataFrames can be combined.

```

temperatures_2022 = pd.Series(
    [3, 6, 9, 13, 18, 21, 24, 23, 19, 14, 8, 4],
    index=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
           'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
)

temperatures_2023 = pd.Series(
    [-3, -1, 4, 9, 15, 20, 20, 19, 16, 15, 7, 6],
    index=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
           'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
)

temperatures_2024 = pd.Series(
    [-1, 2, 5, 8, 17, 24, 25, 20, 17, 14, 9, 2],
    index=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
           'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
)

# Combine Series into DataFrames
df1 = pd.concat([temperatures_2021, temperatures_2022], axis=1)
df2 = pd.concat([temperatures_2023, temperatures_2024], axis=1)

# Combine DataFrames
temperatures = pd.concat([df1, df2], axis=1)
temperatures.columns = [2021, 2022, 2023, 2024]
print(temperatures)

```

	2021	2022	2023	2024
Jan	2	3	-3	-1
Feb	4	6	-1	2
Mar	7	9	4	5
Apr	12	13	9	8
May	19	18	15	17
Jun	23	21	20	24
Jul	25	24	20	25
Aug	23	23	19	20
Sep	18	19	16	17
Oct	15	14	15	14
Nov	9	8	7	9
Dec	5	4	6	2

1.6 Inserting and Deleting in Data Structures

The `del` operator from base Python deletes columns from a DataFrame `del DataFrame['column_name']`. However, Pandas also provides its own methods to add or remove entries row-wise or column-wise.

- `pd.DataFrame.drop(labels=None, axis=0, index=None, columns=None, inplace=False)` removes rows or columns by labels passed as a single value ('Column 1') or as a list (['Column 1', 'Column 2']). The `axis` parameter controls whether rows or columns (`axis=1`) are removed. The `index` and `columns` parameters are alternative ways to pass row or column labels and replace `labels` and `axis`.
- `pd.DataFrame.insert(loc, column, value)` inserts a column at position `loc` with column name `column` and content `value`. If `value` is a Series with a different index, the values can be accessed via `value = Series.values` to insert into the existing index (otherwise, Pandas aligns indices and only inserts matching values).
- Row values can be inserted with `pd.DataFrame.loc[index] = value`. If `value` is a Series, use `value = Series.values` to access its values, otherwise Pandas will try to align the Series index with the DataFrame columns. If a single value is passed, it fills the entire row.

1.7 Exercises: Combining and Deleting

Create an empty DataFrame `df = pd.DataFrame()`.

1. Insert the columns 'len' and 'dose' from the `guinea_pigs` DataFrame.
2. Delete all rows with odd row numbers from the DataFrame `df`.
3. Using the index numbers of `df`, select the corresponding rows from the 'ID' column of the `guinea_pigs` DataFrame. Insert these as a column at index position 0 in `df`.

💡 Tip 5: Example Solution: Combining and Deleting

Exercise 1

```
df = pd.DataFrame()

# Option 1
df['len'] = guinea_pigs['len']

# Option 2
df.insert(loc=1, column='dose', value=guinea_pigs['dose'])

print(df.head(), "\n", df.shape)
```

```
      len  dose
0    4.2   0.5
1   11.5   0.5
2    7.3   0.5
3    5.8   0.5
4    6.4   0.5
(60, 2)
```

Exercise 2

```
df = df.drop(index=range(1, len(df), 2))

print(df.head(), "\n", df.shape)
```

```
      len  dose
0    4.2   0.5
2    7.3   0.5
4    6.4   0.5
6   11.2   0.5
8    5.2   0.5
(30, 2)
```

Exercise 3

```
df.insert(loc=0, column='ID', value=guinea_pigs.loc[df.index, 'ID'])

print(df.head(), "\n")
print(df.tail(), "\n")
print("df.shape:", df.shape)
```

```
ID    len  dose
```

```
0    1    4.2    0.5
2    3    7.3    0.5
4    5    6.4    0.5
6    7   11.2    0.5
8    9    5.2    0.5
```

```
      ID    len  dose
50  51   25.5   2.0
52  53   22.4   2.0
54  55   24.8   2.0
56  57   26.4   2.0
58  59   29.4   2.0
```

```
df.shape: (30, 3)
```

References

https://pandas.pydata.org/docs/user_guide/dsintro.html https://pandas.pydata.org/docs/user_guide/basics.html

2 Operations

Pandas, like NumPy, allows vectorized operations, which means calculations on a Series or DataFrame are applied to each element. This allows arithmetic operators to be used directly.

```
print("Temperatures in Celsius:")
print(27 * "=")
print(temperatures, "\n")

print("Temperatures in Fahrenheit:")
print(27 * "=")
print(temperatures * 9/5 + 32)
```

Temperatures in Celsius:

```
=====
      2021  2022  2023  2024
Jan      2    3   -3   -1
Feb      4    6   -1    2
Mar      7    9    4    5
Apr     12   13    9    8
May     19   18   15   17
Jun     23   21   20   24
Jul     25   24   20   25
Aug     23   23   19   20
Sep     18   19   16   17
Oct     15   14   15   14
Nov      9    8    7    9
Dec      5    4    6    2
```

Temperatures in Fahrenheit:

```
=====
      2021  2022  2023  2024
Jan   35.6  37.4  26.6  30.2
Feb   39.2  42.8  30.2  35.6
Mar   44.6  48.2  39.2  41.0
Apr   53.6  55.4  48.2  46.4
```

May	66.2	64.4	59.0	62.6
Jun	73.4	69.8	68.0	75.2
Jul	77.0	75.2	68.0	77.0
Aug	73.4	73.4	66.2	68.0
Sep	64.4	66.2	60.8	62.6
Oct	59.0	57.2	59.0	57.2
Nov	48.2	46.4	44.6	48.2
Dec	41.0	39.2	42.8	35.6

Boolean operations can also be performed directly.

```
print("Below Zero Temperatures:")
print(27 * "=")
print(temperatures < 0)
```

Below Zero Temperatures:

```
=====
      2021    2022    2023    2024
Jan  False  False   True   True
Feb  False  False   True  False
Mar  False  False  False  False
Apr  False  False  False  False
May  False  False  False  False
Jun  False  False  False  False
Jul  False  False  False  False
Aug  False  False  False  False
Sep  False  False  False  False
Oct  False  False  False  False
Nov  False  False  False  False
Dec  False  False  False  False
```

2.1 Row- and Column-wise Operations

Pandas includes a wide range of methods that perform arithmetic, summary, boolean, and index functions. A complete reference can be found here: <https://pandas.pydata.org/docs/reference/index.html>.

By default, these functions are usually applied column-wise. Using the argument `axis=1` applies the function row-wise. These methods are also available for Series.

Below, some methods are demonstrated as examples.

2.1.1 Arithmetic Functions

The methods `pd.DataFrame.add()`, `pd.DataFrame.sub()`, `pd.DataFrame.mul()`, `pd.DataFrame.div()`, `pd.DataFrame.floordiv()`, `pd.DataFrame.mod()`, and `pd.DataFrame.pow()` correspond to the basic arithmetic operators `+`, `-`, `*`, `/`, `//`, `%`, `**`. They are particularly useful for chained operations.

```
print("Temperatures in Fahrenheit:")
print(27 * "=")
print(temperatures.mul(9).div(5).add(32))
```

```
Temperatures in Fahrenheit:
=====
      2021  2022  2023  2024
Jan   35.6  37.4  26.6  30.2
Feb   39.2  42.8  30.2  35.6
Mar   44.6  48.2  39.2  41.0
Apr   53.6  55.4  48.2  46.4
May   66.2  64.4  59.0  62.6
Jun   73.4  69.8  68.0  75.2
Jul   77.0  75.2  68.0  77.0
Aug   73.4  73.4  66.2  68.0
Sep   64.4  66.2  60.8  62.6
Oct   59.0  57.2  59.0  57.2
Nov   48.2  46.4  44.6  48.2
Dec   41.0  39.2  42.8  35.6
```

Additionally, the `fill_value` parameter can be used to specify a value for missing data (this value is used before the operation). Like NumPy's `np.nan`, Pandas also has a special missing value: `pd.NA` (pay attention to the data type of the output). Handling missing values is covered in detail in the [module on Reading Structured Datasets](#).

```
missing_value = pd.Series([1, pd.NA, 3])
print(missing_value.add(1, fill_value=-999), "\n")
print(missing_value.add(1, fill_value=np.nan), "\n")
print(missing_value.add(1, fill_value=pd.NA))
```

```
0      2
1    -998
2      4
dtype: object
```

```
0      2
1     NaN
2      4
dtype: object
```

```
0      2
1     <NA>
2      4
dtype: object
```

2.1.2 Summary Functions

- `pd.DataFrame.mean()` computes the mean.
- `pd.DataFrame.median()` computes the median.
- `pd.DataFrame.mode()` computes the mode.
- `pd.DataFrame.sum()` computes the sum.
- `pd.DataFrame.cumsum()` computes the cumulative sum.
- `pd.DataFrame.min()` and `pd.DataFrame.max()` compute the minimum and maximum, respectively.
- `pd.DataFrame.cummin()` and `pd.DataFrame.cummax()` compute the cumulative minimum and maximum, respectively.

```
# column-wise
print("Average Annual Temperatures")
print(27 * "=")
print(temperatures.mean(), "\n")

# row-wise
print("Monthly Minimum Temperature")
print(28 * "=")
print(temperatures.min(axis=1))
```

```
Average Annual Temperatures
=====
2021    13.500000
2022    13.500000
2023    10.583333
2024    11.833333
dtype: float64
```

Monthly Minimum Temperature

=====

```
Jan    -3
Feb    -1
Mar     4
Apr     8
May    15
Jun    20
Jul    20
Aug    19
Sep    16
Oct    14
Nov     7
Dec     2
dtype: int64
```

2.1.3 Boolean Functions

Pandas, like base Python, offers various boolean functions.

`pd.DataFrame.isin(values)` checks for each element in the DataFrame whether it is contained in `values`. Using the `~` operator, you can check whether elements of a DataFrame are **not** in `values`: `~pd.DataFrame.isin(values)`.

The behavior of the function depends on the data type of the object passed in `values`.

- If `values` is a list or a NumPy array, the result is True if there is a match with any of the contained elements.
- If `values` is a Series or a DataFrame, the match is checked positionally (see example).

i Note 1: Class-dependent Function Behavior

2.2 Single Values or List

For single values or a list, the match is checked element-wise.

```
print(temperatures, "\n")
print(temperatures.isin([2, 3]))
```

	2021	2022	2023	2024
Jan	2	3	-3	-1
Feb	4	6	-1	2
Mar	7	9	4	5

Apr	12	13	9	8
May	19	18	15	17
Jun	23	21	20	24
Jul	25	24	20	25
Aug	23	23	19	20
Sep	18	19	16	17
Oct	15	14	15	14
Nov	9	8	7	9
Dec	5	4	6	2

	2021	2022	2023	2024
Jan	True	True	False	False
Feb	False	False	False	True
Mar	False	False	False	False
Apr	False	False	False	False
May	False	False	False	False
Jun	False	False	False	False
Jul	False	False	False	False
Aug	False	False	False	False
Sep	False	False	False	False
Oct	False	False	False	False
Nov	False	False	False	False
Dec	False	False	False	True

2.3 NumPy Array

For a NumPy array, the match is checked element-wise (see the next tab for comparison).

```
print(type(temperatures[2021].values), "\n")

print(temperatures.isin(temperatures[2021].values))
```

```
<class 'numpy.ndarray'>
```

	2021	2022	2023	2024
Jan	True	False	False	False
Feb	True	False	False	True
Mar	True	True	True	True
Apr	True	False	True	False
May	True	True	True	False
Jun	True	False	False	False

Jul	True	False	False	True
Aug	True	True	True	False
Sep	True	True	False	False
Oct	True	False	True	False
Nov	True	False	True	True
Dec	True	True	False	True

2.4 Series

For a Series, the match is checked positionally (see the previous tab). The index must match.

```
print(temperatures.isin(temperatures[2021]), "\n")

temperatures_2021_wrong_index = pd.Series([2, 4, 7, 12, 19, 23, 25, 23, 18, 15, 9, 5])
temperatures_2021_wrong_index.index = ['A', 'B', 'C', 'D', 'E', 'F', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

print("The index of the Series is:\n['A', 'B', 'C', 'D', 'E', 'F', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']")
print(temperatures.isin(temperatures_2021_wrong_index))
```

	2021	2022	2023	2024
Jan	True	False	False	False
Feb	True	False	False	False
Mar	True	False	False	False
Apr	True	False	False	False
May	True	False	False	False
Jun	True	False	False	False
Jul	True	False	False	True
Aug	True	True	False	False
Sep	True	False	False	False
Oct	True	False	True	False
Nov	True	False	False	True
Dec	True	False	False	False

The index of the Series is:
 ['A', 'B', 'C', 'D', 'E', 'F', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'].
 The result at index positions A-F is always False.

	2021	2022	2023	2024
Jan	False	False	False	False
Feb	False	False	False	False
Mar	False	False	False	False

Apr	False	False	False	False
May	False	False	False	False
Jun	False	False	False	False
Jul	True	False	False	True
Aug	True	True	False	False
Sep	True	False	False	False
Oct	True	False	True	False
Nov	True	False	False	True
Dec	True	False	False	False

2.5 DataFrame

For a DataFrame, the match is checked positionally. Both the index and column names must match (see the Series tab for index rules).

```
temperatures_2021_df = pd.DataFrame(temperatures[2021])
print(temperatures.isin(temperatures_2021_df), "\n")

temperatures_2021_df.columns = [2035]
print(temperatures.isin(temperatures_2021_df), "\n")
```

	2021	2022	2023	2024
Jan	True	False	False	False
Feb	True	False	False	False
Mar	True	False	False	False
Apr	True	False	False	False
May	True	False	False	False
Jun	True	False	False	False
Jul	True	False	False	False
Aug	True	False	False	False
Sep	True	False	False	False
Oct	True	False	False	False
Nov	True	False	False	False
Dec	True	False	False	False

	2021	2022	2023	2024
Jan	False	False	False	False
Feb	False	False	False	False
Mar	False	False	False	False
Apr	False	False	False	False
May	False	False	False	False

Jun	False	False	False	False
Jul	False	False	False	False
Aug	False	False	False	False
Sep	False	False	False	False
Oct	False	False	False	False
Nov	False	False	False	False
Dec	False	False	False	False

💡 Tip 6: Avoid Surprises

Class-dependent function behavior can distort the results of a data analysis if unnoticed. To prevent this, follow these three general tips:

1. Check the documentation of the respective function. Python and many modules evolve dynamically, so a function's behavior can change.
2. Proceed step by step and use the `print()` function to inspect intermediate results.
3. For large datasets, it is often easier to work with custom test data. A ten-row DataFrame with the same data types and structure as your working dataset is easier to manage. Use such a test dataset to verify the functions you plan to use.

A group of functions implements logical comparisons.

Function	Comparison
<code>pd.DataFrame.lt(other)</code>	less than
<code>pd.DataFrame.le(other)</code>	less than or equal to
<code>pd.DataFrame.eq(other)</code>	equal to
<code>pd.DataFrame.ne(other)</code>	not equal to
<code>pd.DataFrame.ge(other)</code>	greater than or equal to
<code>pd.DataFrame.gt(other)</code>	greater than

```
print(temperatures.le(2), "\n")
print(temperatures[2021].gt(5))
```

	2021	2022	2023	2024
Jan	True	False	True	True
Feb	False	False	True	True
Mar	False	False	False	False
Apr	False	False	False	False
May	False	False	False	False

Jun	False	False	False	False
Jul	False	False	False	False
Aug	False	False	False	False
Sep	False	False	False	False
Oct	False	False	False	False
Nov	False	False	False	False
Dec	False	False	False	True

Jan	False
Feb	False
Mar	True
Apr	True
May	True
Jun	True
Jul	True
Aug	True
Sep	True
Oct	True
Nov	True
Dec	False

Name: 2021, dtype: bool

2.5.1 Using the .agg() and .apply() Methods

Pandas provides two methods to perform operations row- or column-wise. `DataFrame.agg()` (or `DataFrame.aggregate()`) aggregates a `DataFrame` row- or column-wise using a function. The Pandas method `DF.apply()` applies a function row- or column-wise to a `DataFrame`. The methods are very similar and usually produce the same result.

Both functions perform row-wise operations when using the argument `axis=1`.

2.6 Function

```
def my_plus_ten(x):
    y = x + 10
    return y

print(temperatures.agg(my_plus_ten), "\n")
print(temperatures.apply(my_plus_ten))
```

	2021	2022	2023	2024
Jan	12	13	7	9
Feb	14	16	9	12
Mar	17	19	14	15
Apr	22	23	19	18
May	29	28	25	27
Jun	33	31	30	34
Jul	35	34	30	35
Aug	33	33	29	30
Sep	28	29	26	27
Oct	25	24	25	24
Nov	19	18	17	19
Dec	15	14	16	12

	2021	2022	2023	2024
Jan	12	13	7	9
Feb	14	16	9	12
Mar	17	19	14	15
Apr	22	23	19	18
May	29	28	25	27
Jun	33	31	30	34
Jul	35	34	30	35
Aug	33	33	29	30
Sep	28	29	26	27
Oct	25	24	25	24
Nov	19	18	17	19
Dec	15	14	16	12

2.7 Function Name

```
print(temperatures.agg("sum"), "\n")
print(temperatures.apply("sum"))
```

2021	162
2022	162
2023	127
2024	142
dtype: int64	
2021	162

```

2022    162
2023    127
2024    142
dtype: int64

```

2.8 List of Functions

```

print(temperatures.agg(["sum", "mean", "median"]), "\n")
print(temperatures.apply(["sum", "mean", "median"]))

```

	2021	2022	2023	2024
sum	162.0	162.0	127.000000	142.000000
mean	13.5	13.5	10.583333	11.833333
median	13.5	13.5	12.000000	11.500000

	2021	2022	2023	2024
sum	162.0	162.0	127.000000	142.000000
mean	13.5	13.5	10.583333	11.833333
median	13.5	13.5	12.000000	11.500000

2.9 Dictionary of Functions

```

print(temperatures.agg({2021: "sum", 2022: "mean", 2023: "median", 2024: "min"}), "\n")
print(temperatures.apply({2021: "sum", 2022: "mean", 2023: "median", 2024: "min"}), "\n")

```

```

2021    162.0
2022     13.5
2023     12.0
2024     -1.0
dtype: float64

```

```

2021    162.0
2022     13.5
2023     12.0
2024     -1.0
dtype: float64

```

A particularly useful feature is the ability to apply functions that normally operate on a Series to each element of the Series. This uses the lambda syntax: `lambda x: x + 1`. Lambda is a placeholder and can be read as “for each x, do:”. For example, this can be used to determine the number of characters in each element.

```
# Applied to the Series as a whole
print(len(str(temperatures[2021])), "\n")

# Applied element-wise
print(temperatures[2021].agg(lambda x: len(str(x))), "\n") # deprecated
print(temperatures[2021].apply(lambda x: len(str(x))), "\n")
```

144

144

Jan	1
Feb	1
Mar	1
Apr	2
May	2
Jun	2
Jul	2
Aug	2
Sep	2
Oct	2
Nov	1
Dec	1

Name: 2021, dtype: int64

For details on using [lambda expressions](#), refer to the Python documentation.

For completeness, it is worth mentioning that the methods `.map()` and `.transform()` provide very similar alternatives. If interested, you can read about the differences [in this article](#).

```
# print(temperatures[2021].map(lambda x: len(str(x))))
# print(temperatures[2021].transform(lambda x: len(str(x))), "\n")
```

2.10 Exercises: Operations

1. Determine the monthly median temperature for the DataFrame `temperatures`.

2. Identify the months with a median temperature greater than or equal to 21 degrees.
3. Display the index labels of these months.

💡 Tip 7: Sample Solution: Operations Exercises

1. Task

```
print(temperatures.mean(axis=1))
```

```
Jan      0.25
Feb      2.75
Mar      6.25
Apr     10.50
May     17.25
Jun     22.00
Jul     23.50
Aug     21.25
Sep     17.50
Oct     14.50
Nov      8.25
Dec      4.25
dtype: float64
```

2. Task

```
print(temperatures.mean(axis=1).ge(21))
```

```
Jan      False
Feb      False
Mar      False
Apr      False
May      False
Jun       True
Jul       True
Aug       True
Sep      False
Oct      False
Nov      False
Dec      False
dtype: bool
```

3. Task

```
print(temperatures.index[temperatures.mean(axis=1).ge(21)], "\n")

# as a list
print(list(temperatures.index[temperatures.mean(axis=1).ge(21)]), "\n")

Index(['Jun', 'Jul', 'Aug'], dtype='str')

['Jun', 'Jul', 'Aug']
```

2.11 Search and Replace

To determine the index position of a specific value, the NumPy function `np.where()` can be used. It returns two arrays with the corresponding row and column numbers.

```
print(np.where(temperatures == 4))
```

```
(array([ 1,  2, 11]), array([0, 2, 1]))
```

Among other locations, the value 4 can be found in row 1, column 0, or in row 2, column 2.

```
print(temperatures.iloc[1, 0])
print(temperatures.iloc[2, 2])
```

```
4
4
```

Pandas provides two methods to replace values.

- `pd.DataFrame.replace(to_replace, value, *, inplace=False)` replaces `to_replace` with `value`. Using the argument `inplace=True` modifies the object directly.
- `pd.where(cond, other=nan, inplace=False)` keeps values where `cond` is True and replaces all other values with `other` (default is a placeholder for missing values). Using the argument `inplace=True` modifies the object directly.

The syntax of both functions differs slightly, as shown in the example below.

```
print(temperatures.replace(to_replace=25, value=1000), "\n")
print(temperatures.where(temperatures == 25, other=1000))
```

	2021	2022	2023	2024
Jan	2	3	-3	-1
Feb	4	6	-1	2
Mar	7	9	4	5
Apr	12	13	9	8
May	19	18	15	17
Jun	23	21	20	24
Jul	1000	24	20	1000
Aug	23	23	19	20
Sep	18	19	16	17
Oct	15	14	15	14
Nov	9	8	7	9
Dec	5	4	6	2

	2021	2022	2023	2024
Jan	1000	1000	1000	1000
Feb	1000	1000	1000	1000
Mar	1000	1000	1000	1000
Apr	1000	1000	1000	1000
May	1000	1000	1000	1000
Jun	1000	1000	1000	1000
Jul	25	1000	1000	25
Aug	1000	1000	1000	1000
Sep	1000	1000	1000	1000
Oct	1000	1000	1000	1000
Nov	1000	1000	1000	1000
Dec	1000	1000	1000	1000

2.12 Exercises: Search and Replace

1. Determine the positions of values in the DataFrame `temperatures` that are less than 0 and display the values.
2. Replace all values in the DataFrame `temperatures` that are less than 0 with the value 0 and display the result.

💡 Tip 8: Sample Solution: Search and Replace

1. Task

```
print(np.where(temperatures <= 0))
print("Number of values:", len(np.where(temperatures <= 0)[0]))

for i in range(len(np.where(temperatures <= 0)[0])):
    print(temperatures.iloc[np.where(temperatures <= 0)[0][i], np.where(temperatures <= 0)[0][i]])
```

(array([0, 0, 1]), array([2, 3, 2]))
Number of values: 3
-3
-1
-1

2. Task

```
print(temperatures.where(temperatures > 0, other=0))
```

	2021	2022	2023	2024
Jan	2	3	0	0
Feb	4	6	0	2
Mar	7	9	4	5
Apr	12	13	9	8
May	19	18	15	17
Jun	23	21	20	24
Jul	25	24	20	25
Aug	23	23	19	20
Sep	18	19	16	17
Oct	15	14	15	14
Nov	9	8	7	9
Dec	5	4	6	2

2.13 Sorting

The method `DataFrame.sort_index(axis=0, ascending=True, inplace=False)` sorts along an axis, by default in ascending order by the index. Passing the argument `axis=1` sorts the columns. Using `ascending=False` sorts in descending order. The argument `inplace=True` ensures, as usual, that the sorted result is stored directly in the object.

```
print(temperatures.sort_index(), "\n")
print(temperatures.sort_index(axis=1, ascending=False))
```

	2021	2022	2023	2024
Apr	12	13	9	8
Aug	23	23	19	20
Dec	5	4	6	2
Feb	4	6	-1	2
Jan	2	3	-3	-1
Jul	25	24	20	25
Jun	23	21	20	24
Mar	7	9	4	5
May	19	18	15	17
Nov	9	8	7	9
Oct	15	14	15	14
Sep	18	19	16	17

	2024	2023	2022	2021
Jan	-1	-3	3	2
Feb	2	-1	6	4
Mar	5	4	9	7
Apr	8	9	13	12
May	17	15	18	19
Jun	24	20	21	23
Jul	25	20	24	25
Aug	20	19	23	23
Sep	17	16	19	18
Oct	14	15	14	15
Nov	9	7	8	9
Dec	2	6	4	5

The method `DataFrame.sort_values(by, *, axis=0, ascending=True, inplace=False)` sorts values along an axis, by default along the index (`axis=0`). According to the [documentation](#), the `by` parameter should be passed the column name as a string or a list of column names as strings to sort by. However, as shown in the example below, numeric column labels must also be passed in numeric form.

When using the argument `axis=1` to sort along the second dimension, index labels should be provided accordingly.

```
# Sorting by numeric column labels
print(temperatures.sort_values(by=2021), "\n")
print(temperatures.sort_values(by=[2021, 2023]), "\n")

# Sorting by column labels passed as strings
# results in KeyError, the error message is not fully caught
try:
    print(temperatures.sort_values(by='2021'))
except Exception as error:
    print(error)
```

	2021	2022	2023	2024
Jan	2	3	-3	-1
Feb	4	6	-1	2
Dec	5	4	6	2
Mar	7	9	4	5
Nov	9	8	7	9
Apr	12	13	9	8
Oct	15	14	15	14
Sep	18	19	16	17
May	19	18	15	17
Jun	23	21	20	24
Aug	23	23	19	20
Jul	25	24	20	25

	2021	2022	2023	2024
Jan	2	3	-3	-1
Feb	4	6	-1	2
Dec	5	4	6	2
Mar	7	9	4	5
Nov	9	8	7	9
Apr	12	13	9	8
Oct	15	14	15	14
Sep	18	19	16	17
May	19	18	15	17
Aug	23	23	19	20
Jun	23	21	20	24
Jul	25	24	20	25

'2021'

2.14 Exercises: Sorting

1. Sort the DataFrame `guinea_pigs` in descending order by tooth length (`'len'`). Which guinea pig has the longest tooth-forming cell (return the ID)?
2. Which guinea pig that received a dose of 1.0 has the longest tooth-forming cell (return the ID)?

💡 Tip 9: Sample Solution: Sorting

1. Task

```
print(guinea_pigs.sort_values(by='len', ascending=False).head(), "\n")  
print("The ID is:", guinea_pigs.sort_values(by='len', ascending=False).iloc[0, 0])
```

	ID	len	supp	dose
22	23	33.9	VC	2.0
25	26	32.5	VC	2.0
55	56	30.9	OJ	2.0
29	30	29.5	VC	2.0
58	59	29.4	OJ	2.0

The ID is: 23

2. Task

```
dose_1 = guinea_pigs[guinea_pigs['dose'] == 1.0]  
print(dose_1.sort_values(by='len', ascending=False).head(), "\n")  
print("The ID is:", dose_1.sort_values(by='len', ascending=False).iloc[0, 0])
```

	ID	len	supp	dose
49	50	27.3	OJ	1.0
43	44	26.4	OJ	1.0
46	47	25.8	OJ	1.0
45	46	25.2	OJ	1.0
42	43	23.6	OJ	1.0

The ID is: 50

2.15 GroupBy

The method `pd.groupby()` splits a DataFrame (or Series) into groups and returns a GroupBy object. The GroupBy object has the same columns and row labels as the original DataFrame, but is organized according to the grouping. Operations applied to the GroupBy object are executed separately for each group.

This can be observed with the `guinea_pigs` dataset in the following panel.

1. Tab: The dataset contains 60 entries. The first 30 entries in the column 'supp' have the value VC for Vitamin C, and the last 30 entries have the value OJ for orange juice.
2. Tab: Using the method `pd.groupby('supp')`, the dataset can be grouped according to the feature values in the column 'dose' (0.5, 1, and 2).
3. Tab: Operations can be applied to the GroupBy object. For example, the column 'len' can be selected and the mean length of the tooth-forming cell can be determined using the `.mean()` method.
4. Tab: It is also possible to group by multiple features by passing them as a list: `pd.groupby(by=['supp', 'dose'])`.

2.16 DataFrame guinea_pigs

```
print(guinea_pigs.head(n=12))
```

	ID	len	supp	dose
0	1	4.2	VC	0.5
1	2	11.5	VC	0.5
2	3	7.3	VC	0.5
3	4	5.8	VC	0.5
4	5	6.4	VC	0.5
5	6	10.0	VC	0.5
6	7	11.2	VC	0.5
7	8	11.2	VC	0.5
8	9	5.2	VC	0.5
9	10	7.0	VC	0.5
10	11	16.5	VC	1.0
11	12	16.5	VC	1.0

2.17 guinea_pigs grouped by Supplement Type

For the `.head()` method, the argument `n` was halved to display the same number of rows in the output, as this method is applied separately to each of the two groups (VC and OJ).

```
print(guinea_pigs.groupby('supp').head(n=6))
```

	ID	len	supp	dose
0	1	4.2	VC	0.5
1	2	11.5	VC	0.5
2	3	7.3	VC	0.5
3	4	5.8	VC	0.5
4	5	6.4	VC	0.5
5	6	10.0	VC	0.5
30	31	15.2	OJ	0.5
31	32	21.5	OJ	0.5
32	33	17.6	OJ	0.5
33	34	9.7	OJ	0.5
34	35	14.5	OJ	0.5
35	36	10.0	OJ	0.5

2.18 Tooth Length by Supplement Type

```
print(guinea_pigs.groupby(by='supp')['len'].mean())
```

```
supp
OJ    20.663333
VC    16.963333
Name: len, dtype: float64
```

2.19 Tooth Length by Supplement Type and Dose

```
print(guinea_pigs.groupby(by=['supp', 'dose'])['len'].mean())
```

```
supp  dose
OJ    0.5    13.23
      1.0    22.70
```

```

      2.0      26.06
VC    0.5      7.98
      1.0     16.77
      2.0     26.14
Name: len, dtype: float64

```

2.20 Exercises: GroupBy

The Motor Trend Car Road Tests (mtcars) dataset comes from the U.S. magazine Motor Trend from 1974 and contains data for 32 cars.

Code Block 2.1

```

mtcars = pd.read_csv(filepath_or_buffer="01-daten/mtcars.csv", sep=",")
mtcars.rename(columns={'Unnamed: 0': 'car'}, inplace=True)

mtcars.head()

```

	car	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

Column	Description
mpg	Fuel consumption in miles per gallon
cyl	Number of cylinders
disp	Displacement in cubic inches
hp	Horsepower
drat	Rear axle ratio
wt	Weight in 1000 pounds
qsec	Quarter mile time
vs	Engine (0 = V-engine, 1 = straight engine)
am	Transmission (0 = automatic, 1 = manual)
gear	Number of forward gears
carb	Number of carburetors

Henderson and Velleman 1981. Building multiple regression models interactively. *Biometrics* 37: 391–411. The dataset is available on [GitHub](#) and in R.

1. Group the dataset by the number of cylinders and determine the average fuel consumption for each group.
2. Convert the fuel consumption to liters per 100 kilometers.
3. Group the dataset by the number of cylinders and carburetors. Which group is fastest on the quarter mile?

 Tip 10: Sample Solution: GroupBy

1. Task

```
mtcars.groupby(by='cyl')['mpg'].mean()
```

```
cyl
4    26.663636
6    19.742857
8    15.100000
Name: mpg, dtype: float64
```

2. Task

```
# 1 mile = 1.60934 kilometers
# 1 gallon = 3.78541 liters

mpg = mtcars.groupby(by='cyl')['mpg'].mean()

liters_per_100km = 100 / (mpg * 1.60934 / 3.78541)

print(liters_per_100km)
```

```
cyl
4     8.821567
6    11.913932
8    15.577156
Name: mpg, dtype: float64
```

3. Task

```
print(mtcars.groupby(by=['cyl', 'carb'])['qsec'].mean(), "\n")
print(mtcars.groupby(by=['cyl', 'carb'])['qsec'].mean().index[-1], "\n")
```

```
cyl  carb
4    1    19.378000
     2    18.936667
6    1    19.830000
     4    17.670000
     6    15.500000
8    2    17.060000
     3    17.666667
     4    16.495000
     8    14.600000
Name: qsec, dtype: float64
```

```
(np.int64(8), np.int64(8))
```

The group with 8 cylinders and 8 carburetors is the fastest. (Note: This is a so-called [MultiIndex](#).)

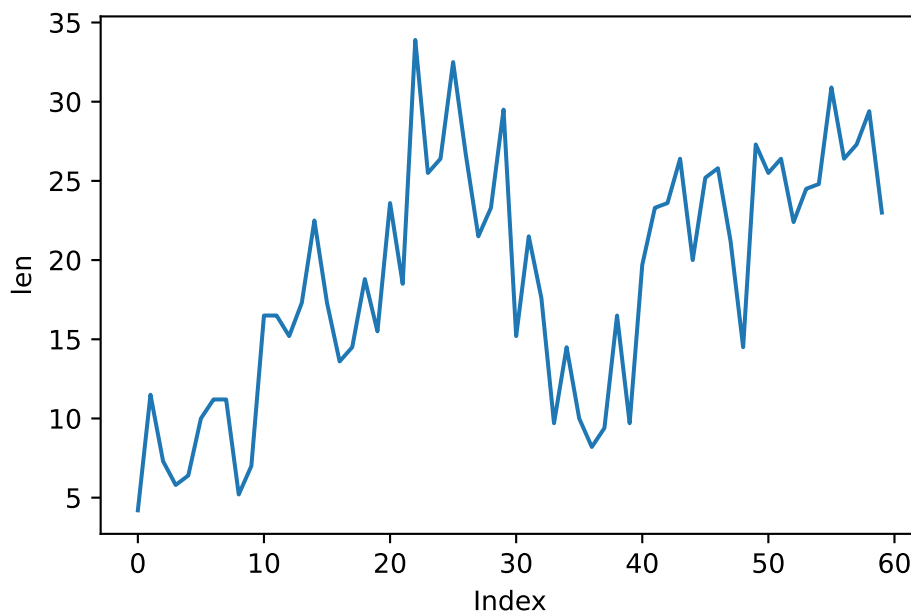
3 Plot Creation

With the method `pd.plot()`, Series and DataFrames can be visualized. Pandas relies on matplotlib for this. The syntax of Pandas is simpler, but the module offers fewer options than matplotlib. However, commands from Pandas and matplotlib can be combined for plot creation (see [Building Block Matplotlib](#)).

3.1 Series

A Series is plotted against its index. By default, a line plot is drawn. The `xlabel` and `ylabel` parameters can be used to add axis labels.

```
guinea_pigs['len'].plot(xlabel = 'Index', ylabel = 'len')
```



The `kind` parameter can be used to change the type of plot. Some options are:

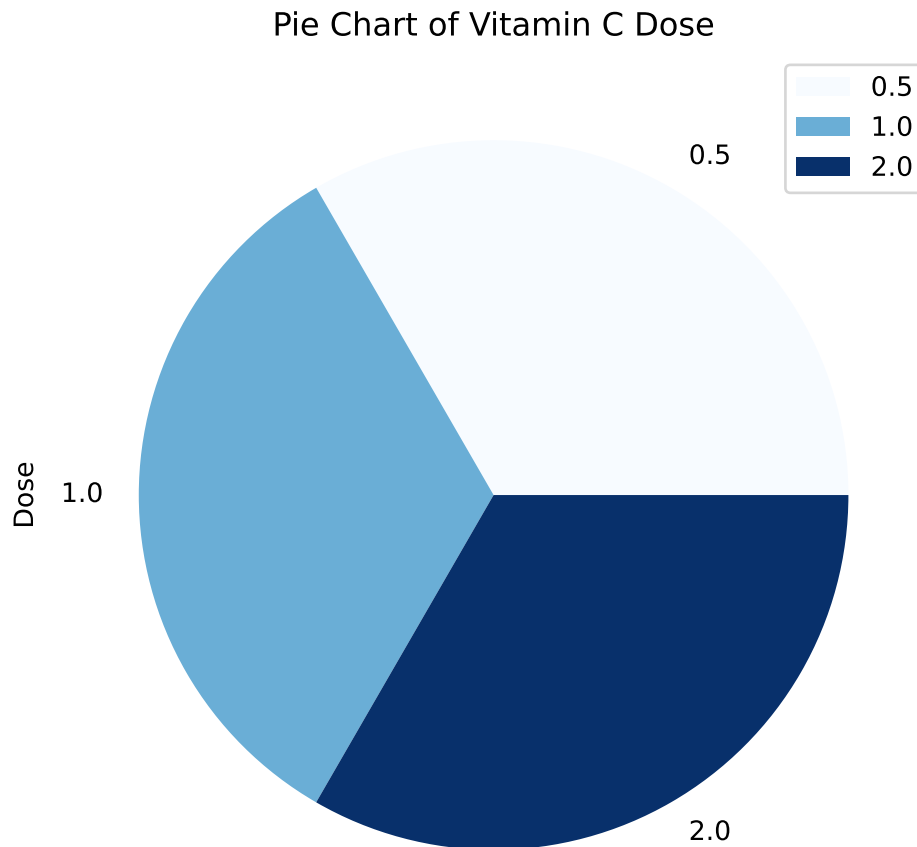
- `'line'`: default line plot

- 'bar': vertical bar chart
- 'hbar': horizontal bar chart
- 'hist': histogram
- 'box': box plot
- 'pie': pie chart

Some useful parameters are:

- `colormap = palette` changes the color palette. A list of palettes available in matplotlib can be found in the [documentation](#).
- `figsize = (width, height)` tuple for figure size in inches
- `legend = True` adds a legend
- `title = 'Title'` adds a title
- `grid = True` adds grid lines
- `xlim = (min, max) / ylim = (min, max)` sets the axis limits

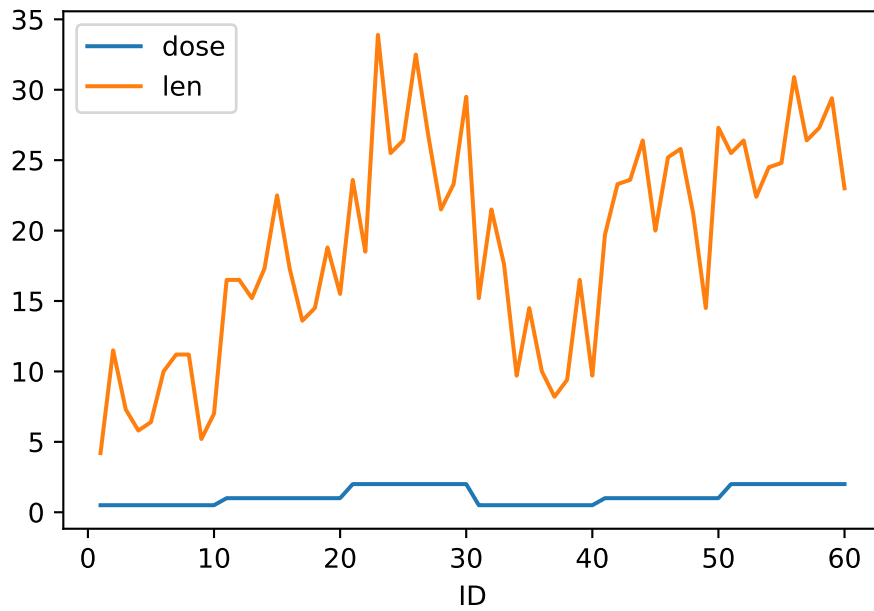
```
guinea_pigs['dose'].value_counts().plot(kind = 'pie', ylabel = 'Dose', colormap = 'Blues', f
```



3.2 DataFrame

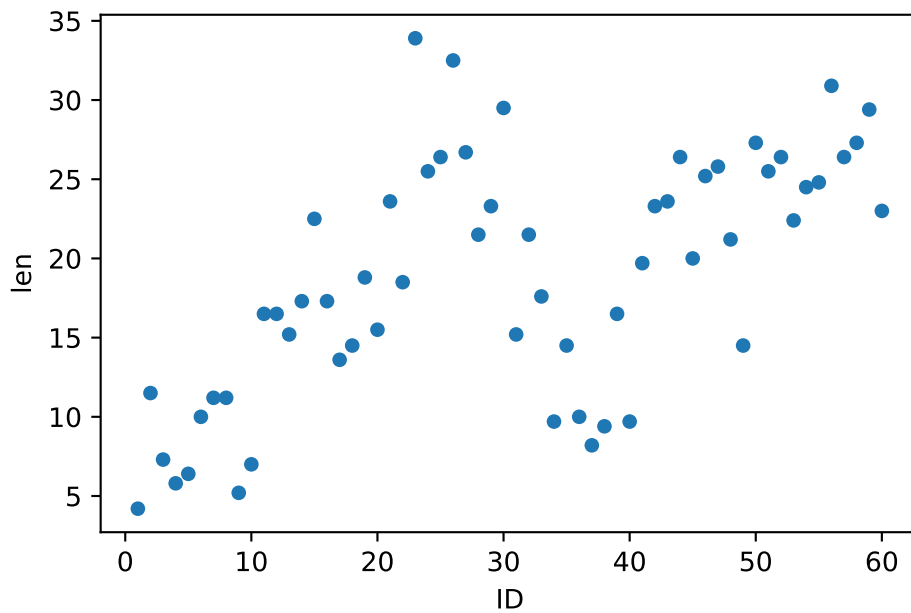
For DataFrames, additional options are available. Using the `x` and `y` parameters, you can select columns or rows to be plotted on the respective axes. `y` can also be a list containing multiple entries.

```
guinea_pigs.plot(x = 'ID', y = ['dose', 'len'])
```



For DataFrames, the scatter plot is available as a plot type.

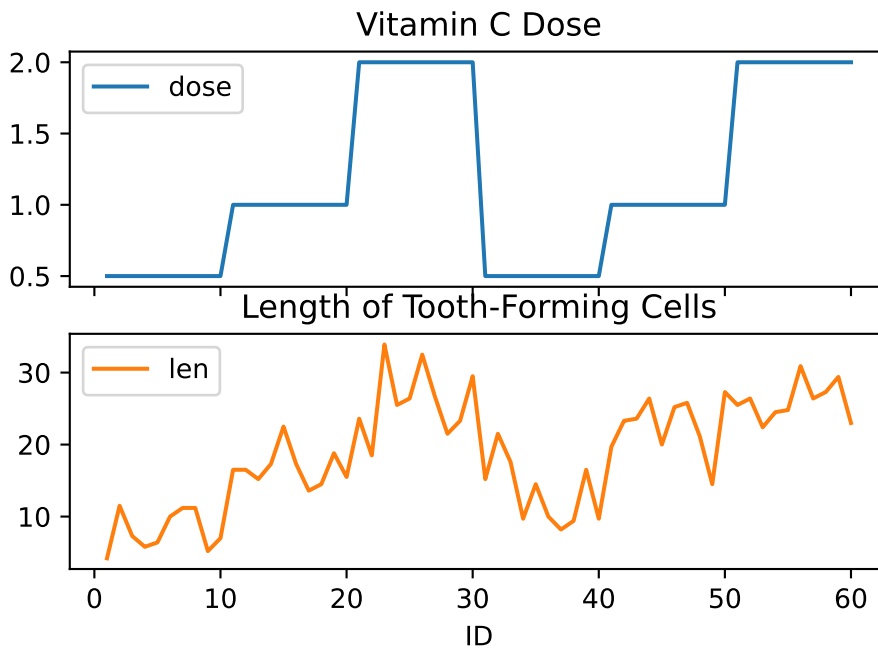
```
guinea_pigs.plot(x = 'ID', y = 'len', kind = 'scatter')
```



3.3 Subplots

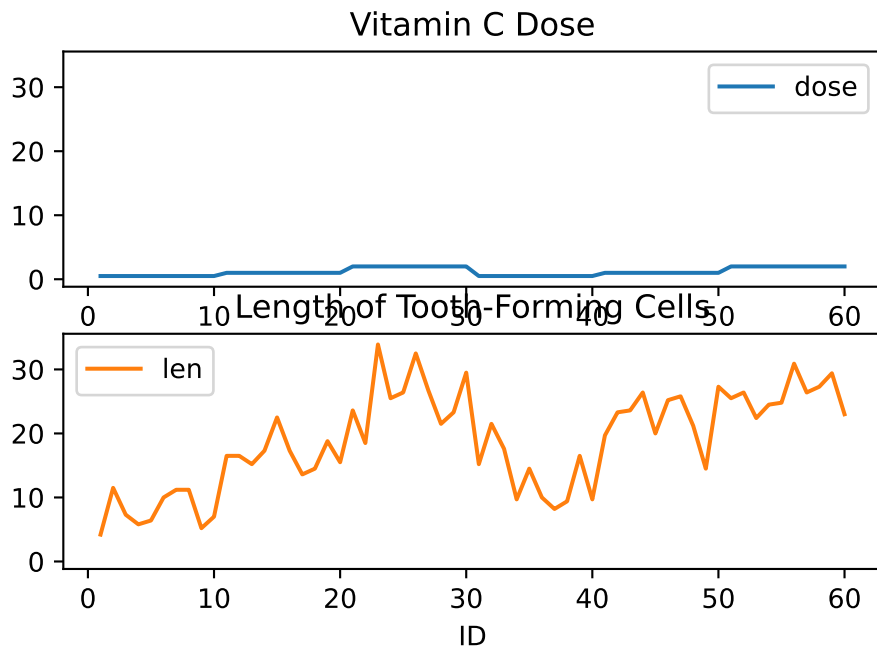
Using the argument `subplots = True` creates separate subplots. The argument `sharex = True` is automatically set, so both subplots share the same x-axis. The `title` parameter can be given a list of titles for each subplot.

```
guinea_pigs.plot(x = 'ID', y = ['dose', 'len'], subplots = True, title = ['Vitamin C Dose',
```



The argument `sharey = True` is also available. The `sharex` argument is set to `False`.

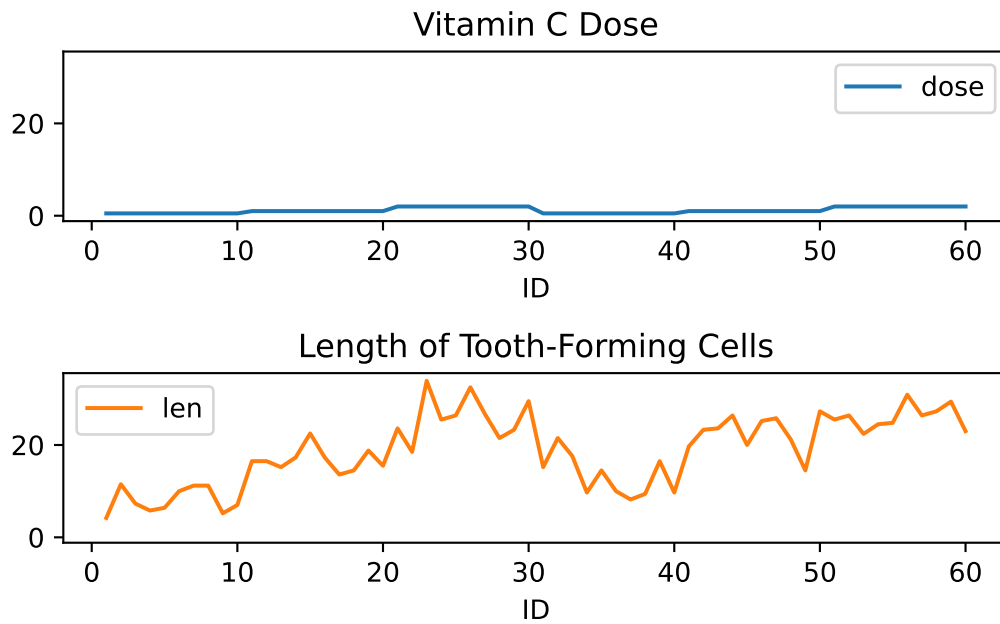
```
guinea_pigs.plot(x = 'ID', y = ['dose', 'len'], subplots = True, sharex = False, sharey = Tr
```



To fix overlapping x-axis labels, a command from the `matplotlib.pyplot` module must be used.

```
import matplotlib.pyplot as plt

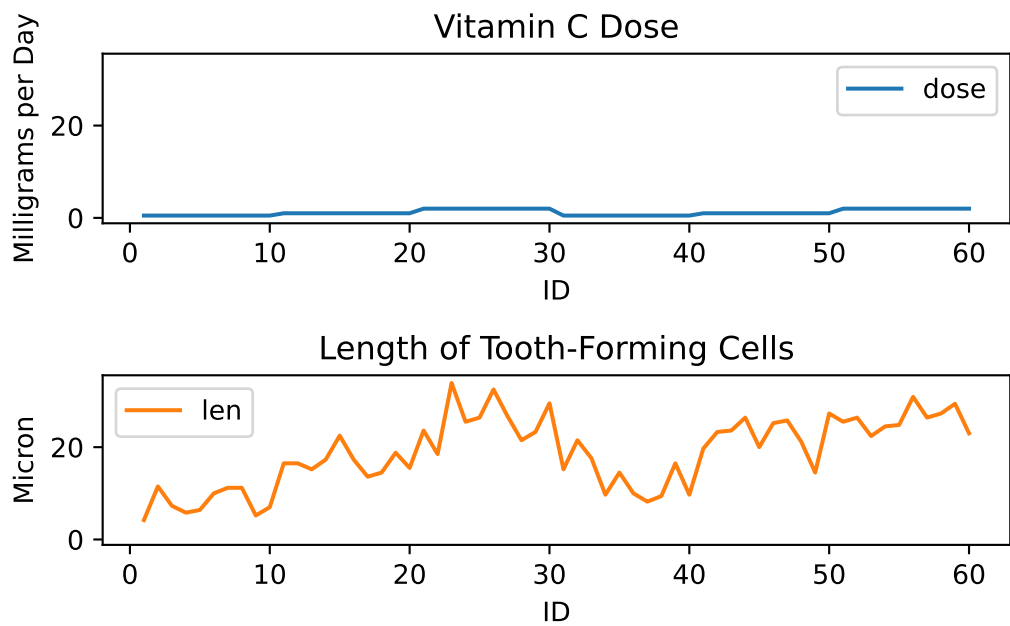
guinea_pigs.plot(x = 'ID', y = ['dose', 'len'], subplots = True, sharex = False, sharey = True)
plt.tight_layout()
```



To set a y-axis label for each subplot, matplotlib must also be used.

```
my_plot = guinea_pigs.plot(x = 'ID', y = ['dose', 'len'], subplots = True, sharex = False, sl

my_plot[0].set_ylabel('Milligrams per Day')
my_plot[1].set_ylabel('Micron')
plt.tight_layout()
plt.show()
```



4 Data Types

The Pandas module is specialized in handling different data types. A DataFrame can contain multiple data types (e.g., numbers and boolean values).

NumPy supports the following data types:

NumPy Array Data Type	Python Data Type
int_	int
double	float
cdouble	complex
bytes_	bytes
str_	str
bool_	bool
datetime64	datetime.datetime
timedelta64	datetime.timedelta

NumPy data types have operating system-dependent equivalents. For example, an array of type `int_` will show as `int`, and an array of floating-point numbers will show as `float64`.

```
scalar = np.array([2])
print(scalar.dtype, "\n")

scalar = np.array([2.1])
print(scalar.dtype)
```

`int64`

`float64`

Additionally, each NumPy data type has a one-letter shorthand. For example, for strings, the shorthand `U` for Unicode characters is used along with the number of characters. (For all other data types, the number represents the number of bytes required in memory.) The output is prefixed with a character indicating the [byte order](#) in memory: `'>'` (big-endian), `'<'` (little-endian), or `'='` (system default).

```

scalar = np.array(['2'])
print(scalar.dtype, "\n")

scalar = np.array(['2.1'])
print(scalar.dtype, "\n")

# A data type with more memory can be assigned
scalar = np.array([2], dtype = 'U3')
print(scalar.dtype)

```

<U1

<U3

<U3

All synonyms and shorthands can be found in the [NumPy documentation](#).

The Pandas module frequently uses NumPy data types, but it also introduces some additional data types. A complete list is available in the [Pandas documentation](#). The most important additional data types are:

- [Categorical](#) dtype = 'category' for categorical, i.e., unordered data.
- [Timezone-aware datetime](#) dtype = 'datetime64[ns, US/Eastern]'
- Extensions of NumPy data types with support for missing values, recognizable by capital letters.

```

# NumPy data type int
series = pd.Series([1, 2, 3], dtype = 'int')
print(series, "\n")

# NumPy int does not support missing values
try:
    series = pd.Series([1, 2, 3, np.nan], dtype = 'int')
except Exception as error:
    print(error, "\n")

# Pandas Int64 supports missing values
series = pd.Series([1, 2, 3, np.nan], dtype = 'Int64')
print(series)

```

```
0    1
1    2
2    3
dtype: int64
```

cannot convert float NaN to integer

```
0      1
1      2
2      3
3    <NA>
dtype: Int64
```

⚠ Warning 4: Pandas Data Type string

Pandas uses the 'string' data type, like Python, which is immutable. This means there is no method that can modify an existing string. Operations with this data type return a new object of type 'string'.

Passing the data type 'str' results in the use of the NumPy string data type (`dtype = 'str'`), which is mutable.

Depending on the situation, using one or the other data type can be useful. For example, the NumPy 'str' type can be concatenated using the `pd.Series.sum()` method.

```
# Using NumPy data type 'str'
string_series = pd.Series(['H', 'a', 'l', 'l', 'o', '!'], dtype = 'str')
print(f"Using NumPy data type 'str': {string_series.sum()}")

# Using Pandas data type 'string'
try:
    string_series.astype('string').sum()
except Exception as error:
    print("\nUsing Pandas data type 'string':")
    print(error)
```

Using NumPy data type 'str': Hallo!

5 Time Series

Processing date and time information in Python is enabled through various modules. A brief introduction to some of these modules is given, as they are occasionally referenced in the documentation. Pandas provides a unified access to most of these functions and uses the NumPy data types `datetime64` and `timedelta64`.

- The data type `datetime64` represents a specific point in time on a specific date and belongs to the `Timestamp` class. The data type has nanosecond precision and can store timezone information.
- The data type `timedelta64` represents an absolute duration of time in nanoseconds and belongs to the `Timedelta` class.

5.1 Date and Time Information in Python

In Python, there are several modules for handling date and time information.

- The `time` module provides time and date operations with objects of type `struct_time`. ([time module documentation](#))
- The `datetime` module introduces the data types `datetime` and `timedelta`, along with additional methods for processing and outputting date and time information. The module can handle years from 1 to 9999 in the Gregorian calendar. ([datetime module documentation](#))
- The `calendar` module provides various calendar functions and extends the range of dates that can be processed. Based on the Gregorian calendar, this extends infinitely in both directions. ([calendar module documentation](#))
- The `pytz` module provides the IANA timezone database (Internet Assigned Numbers Authority, also called the Olsen database) for applications and operating systems. The IANA database includes time zones and changes since 1970. ([Wikipedia](#)) The `pytz` module ensures correct calculation of times at the end of daylight saving time across time zones. ([pytz documentation](#))
- NumPy introduces the data types `datetime64` and `timedelta64`. These are based on the Gregorian calendar and extend infinitely in both directions. <https://numpy.org/doc/stable/reference/arrays.datetime.html>

- Pandas uses the NumPy data types `datetime64` and `timedelta64` and adds many functions for processing date and time information from other packages. https://pandas.pydata.org/docs/user_guide/timeseries.html

NumPy and Pandas can convert datetime objects from other modules into the `datetime64` data type.

5.1.1 Naive and Aware Datetime Objects

Datetime objects are classified as naive or aware depending on whether they contain timezone information. Naive datetime objects lack this information, while aware datetime objects include it. Objects from the `time`, `datetime`, and Pandas modules have a `timezone` attribute, so they are aware. `np.datetime64` has been a naive data type since NumPy version 1.11.0 but still supports timezones for backward compatibility.

“Deprecated since version 1.11.0: NumPy does not store timezone information. For backwards compatibility, `datetime64` still parses timezone offsets, which it handles by converting to `UTC±00:00` (Zulu time). This behaviour is deprecated and will raise an error in the future.” [NumPy Documentation](#)

Timezones

Pandas can handle timezones and convert datetime objects from one timezone to another. The `tz` argument can be used in various functions to specify the timezone.

```
time_series = pd.Series(pd.date_range(start = "2023-03-26T00:00", end = "2023-03-27T00:00",
time_series
```

```
0    2023-03-26 00:00:00+03:00
1    2023-03-26 03:00:00+03:00
2    2023-03-26 06:00:00+03:00
3    2023-03-26 09:00:00+03:00
4    2023-03-26 12:00:00+03:00
5    2023-03-26 15:00:00+03:00
6    2023-03-26 18:00:00+03:00
7    2023-03-26 21:00:00+03:00
8    2023-03-27 00:00:00+03:00
dtype: datetime64[us, Turkey]
```

The function `pd.to_datetime(arg, utc = True)` can be used to convert the timezone to Coordinated Universal Time (UTC).

```
pd.to_datetime(time_series, utc = True)
```

```
0    2023-03-25 21:00:00+00:00
1    2023-03-26 00:00:00+00:00
2    2023-03-26 03:00:00+00:00
3    2023-03-26 06:00:00+00:00
4    2023-03-26 09:00:00+00:00
5    2023-03-26 12:00:00+00:00
6    2023-03-26 15:00:00+00:00
7    2023-03-26 18:00:00+00:00
8    2023-03-26 21:00:00+00:00
dtype: datetime64[us, UTC]
```

Conversion to any desired timezone is possible using the method `pd.Series.dt.tz_convert(tz = 'utc')`.

```
time_series.dt.tz_convert(tz = 'Portugal')
```

```
0    2023-03-25 21:00:00+00:00
1    2023-03-26 00:00:00+00:00
2    2023-03-26 04:00:00+01:00
3    2023-03-26 07:00:00+01:00
4    2023-03-26 10:00:00+01:00
5    2023-03-26 13:00:00+01:00
6    2023-03-26 16:00:00+01:00
7    2023-03-26 19:00:00+01:00
8    2023-03-26 22:00:00+01:00
dtype: datetime64[us, Portugal]
```

Note 2: Listing Available Timezones

The following code lists the timezones available in Python.

```
from zoneinfo import available_timezones

for timezone in sorted(available_timezones()):
    print(timezone)
```

```
Africa/Abidjan
Africa/Accra
```

Africa/Addis_Ababa
Africa/Algiers
Africa/Asmara
Africa/Asmera
Africa/Bamako
Africa/Bangui
Africa/Banjul
Africa/Bissau
Africa/Blantyre
Africa/Brazzaville
Africa/Bujumbura
Africa/Cairo
Africa/Casablanca
Africa/Ceuta
Africa/Conakry
Africa/Dakar
Africa/Dar_es_Salaam
Africa/Djibouti
Africa/Douala
Africa/El_Aaiun
Africa/Freetown
Africa/Gaborone
Africa/Harare
Africa/Johannesburg
Africa/Juba
Africa/Kampala
Africa/Khartoum
Africa/Kigali
Africa/Kinshasa
Africa/Lagos
Africa/Libreville
Africa/Lome
Africa/Luanda
Africa/Lubumbashi
Africa/Lusaka
Africa/Malabo
Africa/Maputo
Africa/Maseru
Africa/Mbabane
Africa/Mogadishu
Africa/Monrovia

Africa/Nairobi
Africa/Ndjamena
Africa/Niamey
Africa/Nouakchott
Africa/Ouagadougou
Africa/Porto-Novo
Africa/Sao_Tome
Africa/Timbuktu
Africa/Tripoli
Africa/Tunis
Africa/Windhoek
America/Adak
America/Anchorage
America/Anguilla
America/Antigua
America/Araguaina
America/Argentina/Buenos_Aires
America/Argentina/Catamarca
America/Argentina/ComodRivadavia
America/Argentina/Cordoba
America/Argentina/Jujuy
America/Argentina/La_Rioja
America/Argentina/Mendoza
America/Argentina/Rio_Gallegos
America/Argentina/Salta
America/Argentina/San_Juan
America/Argentina/San_Luis
America/Argentina/Tucuman
America/Argentina/Ushuaia
America/Aruba
America/Asuncion
America/Atikokan
America/Atka
America/Bahia
America/Bahia_Banderas
America/Barbados
America/Belem
America/Belize
America/Blanc-Sablon
America/Boa_Vista
America/Bogota

America/Boise
America/Buenos_Aires
America/Cambridge_Bay
America/Campo_Grande
America/Cancun
America/Caracas
America/Catamarca
America/Cayenne
America/Cayman
America/Chicago
America/Chihuahua
America/Ciudad_Juarez
America/Coral_Harbour
America/Cordoba
America/Costa_Rica
America/Coyhaique
America/Creston
America/Cuiaba
America/Curacao
America/Danmarkshavn
America/Dawson
America/Dawson_Creek
America/Denver
America/Detroit
America/Dominica
America/Edmonton
America/Eirunepe
America/El_Salvador
America/Ensenada
America/Fort_Nelson
America/Fort_Wayne
America/Fortaleza
America/Glace_Bay
America/Godthab
America/Goose_Bay
America/Grand_Turk
America/Grenada
America/Guadeloupe
America/Guatemala
America/Guayaquil
America/Guyana

America/Halifax
America/Havana
America/Hermosillo
America/Indiana/Indianapolis
America/Indiana/Knox
America/Indiana/Marengo
America/Indiana/Petersburg
America/Indiana/Tell_City
America/Indiana/Vevay
America/Indiana/Vincennes
America/Indiana/Winamac
America/Indianapolis
America/Inuvik
America/Iqaluit
America/Jamaica
America/Jujuy
America/Juneau
America/Kentucky/Louisville
America/Kentucky/Monticello
America/Knox_IN
America/Kralendijk
America/La_Paz
America/Lima
America/Los_Angeles
America/Louisville
America/Lower_Princes
America/Maceio
America/Managua
America/Manaus
America/Marigot
America/Martinique
America/Matamoros
America/Mazatlan
America/Mendoza
America/Menominee
America/Merida
America/Metlakatla
America/Mexico_City
America/Miquelon
America/Moncton
America/Monterrey

America/Montevideo
America/Montreal
America/Montserrat
America/Nassau
America/New_York
America/Nipigon
America/Nome
America/Noronha
America/North_Dakota/Beulah
America/North_Dakota/Center
America/North_Dakota/New_Salem
America/Nuuk
America/Ojinaga
America/Panama
America/Pangnirtung
America/Paramaribo
America/Phoenix
America/Port-au-Prince
America/Port_of_Spain
America/Porto_Acre
America/Porto_Velho
America/Puerto_Rico
America/Punta_Arenas
America/Rainy_River
America/Rankin_Inlet
America/Recife
America/Regina
America/Resolute
America/Rio_Branco
America/Rosario
America/Santa_Isabel
America/Santarem
America/Santiago
America/Santo_Domingo
America/Sao_Paulo
America/Scoresbysund
America/Shiprock
America/Sitka
America/St_Barthelemy
America/St_Johns
America/St_Kitts

America/St_Lucia
America/St_Thomas
America/St_Vincent
America/Swift_Current
America/Tegucigalpa
America/Thule
America/Thunder_Bay
America/Tijuana
America/Toronto
America/Tortola
America/Vancouver
America/Virgin
America/Whitehorse
America/Winnipeg
America/Yakutat
America/Yellowknife
Antarctica/Casey
Antarctica/Davis
Antarctica/DumontDUrville
Antarctica/Macquarie
Antarctica/Mawson
Antarctica/McMurdo
Antarctica/Palmer
Antarctica/Rothera
Antarctica/South_Pole
Antarctica/Syowa
Antarctica/Troll
Antarctica/Vostok
Arctic/Longyearbyen
Asia/Aden
Asia/Almaty
Asia/Amman
Asia/Anadyr
Asia/Aqtan
Asia/Aqtobe
Asia/Ashgabat
Asia/Ashkhabad
Asia/Atyrau
Asia/Baghdad
Asia/Bahrain
Asia/Baku

Asia/Bangkok
Asia/Barnaul
Asia/Beirut
Asia/Bishkek
Asia/Brunei
Asia/Calcutta
Asia/Chita
Asia/Choibalsan
Asia/Chongqing
Asia/Chungking
Asia/Colombo
Asia/Dacca
Asia/Damascus
Asia/Dhaka
Asia/Dili
Asia/Dubai
Asia/Dushanbe
Asia/Famagusta
Asia/Gaza
Asia/Harbin
Asia/Hebron
Asia/Ho_Chi_Minh
Asia/Hong_Kong
Asia/Hovd
Asia/Irkutsk
Asia/Istanbul
Asia/Jakarta
Asia/Jayapura
Asia/Jerusalem
Asia/Kabul
Asia/Kamchatka
Asia/Karachi
Asia/Kashgar
Asia/Kathmandu
Asia/Katmandu
Asia/Khandyga
Asia/Kolkata
Asia/Krasnoyarsk
Asia/Kuala_Lumpur
Asia/Kuching
Asia/Kuwait

Asia/Macao
Asia/Macau
Asia/Magadan
Asia/Makassar
Asia/Manila
Asia/Muscat
Asia/Nicosia
Asia/Novokuznetsk
Asia/Novosibirsk
Asia/Omsk
Asia/Oral
Asia/Phnom_Penh
Asia/Pontianak
Asia/Pyongyang
Asia/Qatar
Asia/Qostanay
Asia/Qyzylorda
Asia/Rangoon
Asia/Riyadh
Asia/Saigon
Asia/Sakhalin
Asia/Samarkand
Asia/Seoul
Asia/Shanghai
Asia/Singapore
Asia/Srednekolymsk
Asia/Taipei
Asia/Tashkent
Asia/Tbilisi
Asia/Tehran
Asia/Tel_Aviv
Asia/Thimbu
Asia/Thimphu
Asia/Tokyo
Asia/Tomsk
Asia/Ujung_Pandang
Asia/Ulaanbaatar
Asia/Ulan_Bator
Asia/Urumqi
Asia/Ust-Nera
Asia/Vientiane

Asia/Vladivostok
Asia/Yakutsk
Asia/Yangon
Asia/Yekaterinburg
Asia/Yerevan
Atlantic/Azores
Atlantic/Bermuda
Atlantic/Canary
Atlantic/Cape_Verde
Atlantic/Faeroe
Atlantic/Faroe
Atlantic/Jan_Mayen
Atlantic/Madeira
Atlantic/Reykjavik
Atlantic/South_Georgia
Atlantic/St_Helena
Atlantic/Stanley
Australia/ACT
Australia/Adelaide
Australia/Brisbane
Australia/Broken_Hill
Australia/Canberra
Australia/Currie
Australia/Darwin
Australia/Eucla
Australia/Hobart
Australia/LHI
Australia/Lindeman
Australia/Lord_Howe
Australia/Melbourne
Australia/NSW
Australia/North
Australia/Perth
Australia/Queensland
Australia/South
Australia/Sydney
Australia/Tasmania
Australia/Victoria
Australia/West
Australia/Yancowinna
Brazil/Acre

Brazil/DeNoronha
Brazil/East
Brazil/West
CET
CST6CDT
Canada/Atlantic
Canada/Central
Canada/Eastern
Canada/Mountain
Canada/Newfoundland
Canada/Pacific
Canada/Saskatchewan
Canada/Yukon
Chile/Continental
Chile/EasterIsland
Cuba
EET
EST
EST5EDT
Egypt
Eire
Etc/GMT
Etc/GMT+0
Etc/GMT+1
Etc/GMT+10
Etc/GMT+11
Etc/GMT+12
Etc/GMT+2
Etc/GMT+3
Etc/GMT+4
Etc/GMT+5
Etc/GMT+6
Etc/GMT+7
Etc/GMT+8
Etc/GMT+9
Etc/GMT-0
Etc/GMT-1
Etc/GMT-10
Etc/GMT-11
Etc/GMT-12
Etc/GMT-13

Etc/GMT-14
Etc/GMT-2
Etc/GMT-3
Etc/GMT-4
Etc/GMT-5
Etc/GMT-6
Etc/GMT-7
Etc/GMT-8
Etc/GMT-9
Etc/GMT0
Etc/Greenwich
Etc/UCT
Etc/UTC
Etc/Universal
Etc/Zulu
Europe/Amsterdam
Europe/Andorra
Europe/Astrakhan
Europe/Athens
Europe/Belfast
Europe/Belgrade
Europe/Berlin
Europe/Bratislava
Europe/Brussels
Europe/Bucharest
Europe/Budapest
Europe/Busingen
Europe/Chisinau
Europe/Copenhagen
Europe/Dublin
Europe/Gibraltar
Europe/Guernsey
Europe/Helsinki
Europe/Isle_of_Man
Europe/Istanbul
Europe/Jersey
Europe/Kaliningrad
Europe/Kiev
Europe/Kirov
Europe/Kyiv
Europe/Lisbon

Europe/Ljubljana
Europe/London
Europe/Luxembourg
Europe/Madrid
Europe/Malta
Europe/Mariehamn
Europe/Minsk
Europe/Monaco
Europe/Moscow
Europe/Nicosia
Europe/Oslo
Europe/Paris
Europe/Podgorica
Europe/Prague
Europe/Riga
Europe/Rome
Europe/Samara
Europe/San_Marino
Europe/Sarajevo
Europe/Saratov
Europe/Simferopol
Europe/Skopje
Europe/Sofia
Europe/Stockholm
Europe/Tallinn
Europe/Tirane
Europe/Tiraspol
Europe/Ulyanovsk
Europe/Uzhgorod
Europe/Vaduz
Europe/Vatican
Europe/Vienna
Europe/Vilnius
Europe/Volgograd
Europe/Warsaw
Europe/Zagreb
Europe/Zaporozhye
Europe/Zurich
Factory
GB
GB-Eire

GMT
GMT+0
GMT-0
GMT0
Greenwich
HST
Hongkong
Iceland
Indian/Antananarivo
Indian/Chagos
Indian/Christmas
Indian/Cocos
Indian/Comoro
Indian/Kerguelen
Indian/Mahe
Indian/Maldives
Indian/Mauritius
Indian/Mayotte
Indian/Reunion
Iran
Israel
Jamaica
Japan
Kwajalein
Libya
MET
MST
MST7MDT
Mexico/BajaNorte
Mexico/BajaSur
Mexico/General
NZ
NZ-CHAT
Navajo
PRC
PST8PDT
Pacific/Apia
Pacific/Auckland
Pacific/Bougainville
Pacific/Chatham
Pacific/Chuuk

Pacific/Easter
Pacific/Efate
Pacific/Enderbury
Pacific/Fakaofu
Pacific/Fiji
Pacific/Funafuti
Pacific/Galapagos
Pacific/Gambier
Pacific/Guadalcanal
Pacific/Guam
Pacific/Honolulu
Pacific/Johnston
Pacific/Kanton
Pacific/Kiritimati
Pacific/Kosrae
Pacific/Kwajalein
Pacific/Majuro
Pacific/Marquesas
Pacific/Midway
Pacific/Nauru
Pacific/Niue
Pacific/Norfolk
Pacific/Noumea
Pacific/Pago_Pago
Pacific/Palau
Pacific/Pitcairn
Pacific/Pohnpei
Pacific/Ponape
Pacific/Port_Moresby
Pacific/Rarotonga
Pacific/Saipan
Pacific/Samoa
Pacific/Tahiti
Pacific/Tarawa
Pacific/Tongatapu
Pacific/Truk
Pacific/Wake
Pacific/Wallis
Pacific/Yap
Poland
Portugal

```
ROC
ROK
Singapore
Turkey
UCT
US/Alaska
US/Aleutian
US/Arizona
US/Central
US/East-Indiana
US/Eastern
US/Hawaii
US/Indiana-Starke
US/Michigan
US/Mountain
US/Pacific
US/Samoa
UTC
Universal
W-SU
WET
Zulu
localtime
```

5.1.2 Everything is Relative: The Epoch

Python stores time relative to a reference point, the Unix time, known as the epoch. The epoch can be displayed using the function `pd.to_datetime(0)`. This function converts arguments into timestamps. Integers are interpreted as nanoseconds since the epoch. We will cover this function in more detail later.

```
import pandas as pd
print(pd.to_datetime(0))
```

```
1970-01-01 00:00:00
```

⚠ Warning 5: Time – Atomic, Coordinated, or Universal?

NumPy uses International Atomic Time (abbreviated TAI for French *Temps Atomique International*). TAI assumes each calendar day is exactly 86,400 seconds long, so it does not account for leap seconds. Atomic time forms the basis for Coordinated Universal Time (UTC).

UTC stands for Coordinated Universal Time (also known as Greenwich Mean Time). The abbreviation UTC is a compromise between English and French. Coordinated Universal Time compensates for the slowing of Earth's rotation (astronomically measured as Universal Time, UT) with leap seconds to account for the slight lengthening of a day. TAI is therefore ahead of UTC. Since 1972, the difference between TAI and UTC is an integer number of seconds. Currently (2024), TAI is 37 seconds ahead of UTC.

A conversion to Coordinated Universal Time is not yet implemented in NumPy. ([NumPy documentation](#), [Wikipedia](#)).

5.1.3 Daylight Saving Time

“DST is Daylight Saving Time, an adjustment of the timezone by (usually) one hour during part of the year. DST rules are magic (determined by local law) and can change from year to year. The C library has a table containing the local rules (often it is read from a system file for flexibility) and is the only source of True Wisdom in this respect.” ([time module documentation](#))

Pandas works with Coordinated Universal Time (UTC) by default. UTC itself is not a timezone and therefore does not observe daylight saving time. Daylight saving is applied depending on the timezone. For example, Turkey abolished daylight saving in 2016 (and permanently adopted summer time).

In the following examples, a 24-hour time delta is added at 9 AM on the day before the DST change. Since the clock moves forward one hour during the night (morning of the next day), the resulting timestamp shows 10 AM if DST is in effect.

```
print("No daylight saving in UTC:")
print(pd.Timestamp("2025-03-29T09:00") + pd.Timedelta(24, "h"), "\n")

print("Timezone with daylight saving:")
print(pd.Timestamp("2025-03-29T09:00", tz="Europe/Berlin") + pd.Timedelta(24, "h"), "\n")

print("No daylight saving in Turkey today:")
```

```
print(pd.Timestamp("2025-03-29T09:00", tz="Turkey") + pd.Timedelta(24, "h"), "\n")

print("Turkey before abolishing daylight saving:")
print(pd.Timestamp("2014-03-30T09:00", tz="Turkey") + pd.Timedelta(24, "h"))
```

No daylight saving in UTC:
2025-03-30 09:00:00

Timezone with daylight saving:
2025-03-30 10:00:00+02:00

No daylight saving in Turkey today:
2025-03-30 09:00:00+03:00

Turkey before abolishing daylight saving:
2014-03-31 10:00:00+03:00

A list of timezones can be found on Wikipedia: https://en.wikipedia.org/wiki/List_of_tz_database_time_zones

5.1.4 Calendar

The `calendar`, `NumPy`, and `Pandas` modules use the Gregorian calendar extended backward before its introduction in 1582, known as the [proleptic Gregorian calendar](#). While the `date` module only supports years 1–9999 CE, the `datetime64` data type can also handle years before the Common Era using [astronomical year numbering](#). This means there is a year 0 (the first year before CE), and preceding years are represented with negative numbers (-1 is the second year BCE). [NumPy Documentation](#)

5.2 datetime in Pandas

Pandas uses the `NumPy` data type `datetime64` to process date and time information. In Pandas, `datetime64` objects are created using the functions `pd.to_datetime()` or `pd.date_range()`. *Note: Another option is the `pd.Timestamp()` function, which provides more extensive options for creating a timestamp but does not support string parsing.*

`pd.to_datetime()` generates values of type `datetime64[ns]` (scalars created with `pd.to_datetime()` are returned as `Timestamp` objects, which do not have a `dtype` attribute). The function accepts the following inputs:

- datetime objects from other modules.
- Numbers and a time unit `pd.to_datetime(1, unit=None)` (default is nanoseconds). The `unit` argument accepts 'ns', 'ms', 's', 'm', 'h', 'D', 'W', 'M', 'Y' for nanosecond, millisecond, second, minute, hour, day, week, month, or year. The result is a timestamp relative to the epoch.

```
print(pd.to_datetime(1000, unit = 'D'))
print(pd.to_datetime(1000 * 1000, unit = 'h'))
print(pd.to_datetime(1000 * 1000 * 1000, unit = 's'))
```

```
1972-09-27 00:00:00
2084-01-29 16:00:00
2001-09-09 01:46:40
```

- Strings representing a date or a date with time, formatted according to [ISO 8601](#).

```
print(pd.to_datetime('2017'))
print(pd.to_datetime('2017-01-01T00'))
print(pd.to_datetime('2017-01-01 00:00:00'))
```

```
2017-01-01 00:00:00
2017-01-01 00:00:00
2017-01-01 00:00:00
```

- Strings in other formats using the argument `format = "%d/%m/%Y"` (see [strftime documentation for string formatting](#)).

```
print(pd.to_datetime('Monday, 12. August `24', format = "%A, %d. %B `%y"))
print(pd.to_datetime('Monday, 12. August 2024, 12:15 Uhr CET', format = "%A, %d. %B %Y, %H:%M"))
```

```
2024-08-12 00:00:00
2024-08-12 12:15:00+02:00
```

- Dictionary or DataFrame.

```
print(pd.to_datetime({'year':[2020, 2024], 'month': [1, 11], 'day': [1, 21]}), "\n")
print(pd.to_datetime(pd.DataFrame({'year':[2020, 2024], 'month': [1, 11], 'day': [1, 21]})))
```

```
0    2020-01-01
1    2024-11-21
dtype: datetime64[us]
```

```
0    2020-01-01
1    2024-11-21
dtype: datetime64[us]
```

The function `pd.date_range()` generates an array of type `DatetimeIndex` with dtype `datetime64`. Exactly three of the following four arguments are required to create a range:

- **start**: start of the range.
- **end**: end of the range (inclusive).
- **freq**: step size (e.g., year, day, business day, hour, or multiples such as ‘6h’ — see [list of available strings](#)).
- **periods**: number of values to generate.

```
print(pd.date_range(start = '2017', end = '2024', periods = 3), "\n")
print(pd.date_range(start = '2017', end = '2024', freq = 'YE'), "\n")
print(pd.date_range(end = '2024', freq = 'h', periods = 3))
```

```
DatetimeIndex(['2017-01-01', '2020-07-02', '2024-01-01'], dtype='datetime64[us]', freq=None)
```

```
DatetimeIndex(['2017-12-31', '2018-12-31', '2019-12-31', '2020-12-31',
               '2021-12-31', '2022-12-31', '2023-12-31'],
              dtype='datetime64[us]', freq='YE-DEC')
```

```
DatetimeIndex(['2023-12-31 22:00:00', '2023-12-31 23:00:00',
               '2024-01-01 00:00:00'],
              dtype='datetime64[us]', freq='h')
```

Warning 6: `pd.date_range()`

The function `pd.date_range()` will no longer support the abbreviation ‘Y’ in the future. Instead, the abbreviations ‘YS’ (year start) or ‘YE’ (year end) can be used. Similarly, ‘M’ will be replaced by ‘MS’ (month start) or ‘ME’ (month end).

5.3 timedelta in Pandas

Time differences are created using the function `pd.Timedelta()`. Time differences can be created either by specifying an integer and a time unit or by using keyword arguments (valid arguments are: weeks, days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds).

```
print(pd.Timedelta(1, 'D'))
print(pd.Timedelta(days = 1, hours = 1))
```

```
1 days 00:00:00
1 days 01:00:00
```

Important: Unlike in NumPy, time differences in months and years are no longer supported by Pandas.

```
try:
    print(pd.Timedelta(1, 'Y'))
except ValueError as error:
    print(error)
else:
    print(pd.Timedelta(1, 'Y'))
```

Units 'M', 'Y', and 'y' are no longer supported, as they do not represent unambiguous time differences.

Time differences can also be created using a string.

```
print(pd.Timedelta('10sec'))
print(pd.Timedelta('10min'))
print(pd.Timedelta('10hours'))
print(pd.Timedelta('10days'))
print(pd.Timedelta('10w'))
```

```
0 days 00:00:10
0 days 00:10:00
0 days 10:00:00
10 days 00:00:00
70 days 00:00:00
```

```
/tmp/ipykernel_3790/3776135813.py:5: Pandas4Warning: 'w' is deprecated and will be removed in
print(pd.Timedelta('10w'))
```

Time series can be easily shifted using a time difference.

```
pd.date_range(start = '2024-01-01T00:00', end = '2024-01-01T02:00', freq = '15min') + pd.Timedelta('10w')
```

```
DatetimeIndex(['2024-01-01 00:30:00', '2024-01-01 00:45:00',
               '2024-01-01 01:00:00', '2024-01-01 01:15:00',
               '2024-01-01 01:30:00', '2024-01-01 01:45:00',
               '2024-01-01 02:00:00', '2024-01-01 02:15:00',
               '2024-01-01 02:30:00'],
              dtype='datetime64[us]', freq='15min')
```

5.4 Accessing Time Series

Pandas provides numerous attributes and methods to extract information from `datetime64` objects. An overview of all available attributes and methods can be obtained with `dir(pd.to_datetime(0))` or by the example code below.

```
# Attributes
print("Year:", pd.to_datetime(0).year)
print("Month:", pd.to_datetime(0).month)
print("Day:", pd.to_datetime(0).day)
print("Hour:", pd.to_datetime(0).hour)
print("Minute:", pd.to_datetime(0).minute)
print("Second:", pd.to_datetime(0).second)
print("Day of the year:", pd.to_datetime(0).dayofyear)
print("Day of the week:", pd.to_datetime(0).dayofweek)
print("Days in month:", pd.to_datetime(0).days_in_month)
print("Leap year:", pd.to_datetime(0).is_leap_year)

# Methods
print("\nDate:", pd.to_datetime(0).date())
print("Time:", pd.to_datetime(0).time())
print("Weekday (0-6):", pd.to_datetime(0).weekday())
print("Month name:", pd.to_datetime(0).month_name())
```

Year: 1970

Month: 1

Day: 1
Hour: 0
Minute: 0
Second: 0
Day of the year: 1
Day of the week: 3
Days in month: 31
Leap year: False

Date: 1970-01-01
Time: 00:00:00
Weekday (0-6): 3
Month name: January

i Note 3: Attributes and Methods of a datetime Object

```
obj = pd.to_datetime(0)

attributes = [attr for attr in dir(obj) if not (callable(getattr(obj, attr)) or attr.startswith('_'))]
print("Attributes:")
print(30 * "=")
print(attributes)

methods = [attr for attr in dir(obj) if (callable(getattr(obj, attr)) and not attr.startswith('_'))]
print("\nMethods:")
print(30 * "=")
print(methods)
```

```
Attributes:
=====
['asm8', 'day', 'day_of_week', 'day_of_year', 'dayofweek', 'dayofyear', 'days_in_month', 'days_in_year', 'is_leap_year', 'month', 'month_name', 'month_name_full', 'month_name_short', 'quarter', 'quarter_name', 'quarter_name_full', 'quarter_name_short', 'second', 'time', 'year']

Methods:
=====
['_from_dt64', '_from_value_and_reso', '_round', 'as_unit', 'astimezone', 'ceil', 'combine', 'floor', 'strftime', 'to_datetime', 'to_pydatetime', 'total_seconds', 'tz_convert', 'tz_localize', 'tzinfo', 'unit', 'year']
```

For `pd.Series`, access is done via the `.dt` accessor (see [.dt accessor](#)). Accessing different information via an attribute (without parentheses) or a method (with parentheses) can differ (see the example below).

The dt Accessor

```
# Attributes
print("Date:", pd.Series(pd.to_datetime(0)).dt.date) # Difference
print("Time:", pd.Series(pd.to_datetime(0)).dt.time) # Difference
print("Year:", pd.Series(pd.to_datetime(0)).dt.year)
print("Month:", pd.Series(pd.to_datetime(0)).dt.month)
print("Day:", pd.Series(pd.to_datetime(0)).dt.day)
print("Hour:", pd.Series(pd.to_datetime(0)).dt.hour)
print("Minute:", pd.Series(pd.to_datetime(0)).dt.minute)
print("Second:", pd.Series(pd.to_datetime(0)).dt.second)

print("\nDay of the year:", pd.Series(pd.to_datetime(0)).dt.dayofyear)
print("Day of the week:", pd.Series(pd.to_datetime(0)).dt.dayofweek)
print("Weekday:", pd.Series(pd.to_datetime(0)).dt.weekday) # Difference
print("Days in month:", pd.Series(pd.to_datetime(0)).dt.days_in_month)
print("Leap year:", pd.Series(pd.to_datetime(0)).dt.is_leap_year)

# Methods
print("\nMonth name:", pd.Series(pd.to_datetime(0)).dt.month_name())
```

Date: 0 1970-01-01

dtype: object

Time: 0 00:00:00

dtype: object

Year: 0 1970

dtype: int32

Month: 0 1

dtype: int32

Day: 0 1

dtype: int32

Hour: 0 0

dtype: int32

Minute: 0 0

dtype: int32

Second: 0 0

dtype: int32

Day of the year: 0 1

dtype: int32

Day of the week: 0 3

dtype: int32

```
Weekday: 0      3
dtype: int32
Days in month: 0      31
dtype: int32
Leap year: 0      False
dtype: bool

Month name: 0      January
dtype: str
```

5.5 Exercises

1. How old are you in days? How old in seconds?
2. What day of the week was your birthday?
3. How many days are left until Christmas?
4. Create a list of all leap years in the 20th century.

Tip 11: Sample Solution

Exercise 1

Replace the string 'YYYY-MM-DD' or, if you know the time of your birth, 'YYYY-MM-DDTHH:MM' with your birthday.

In Pandas, the keywords `pd.to_datetime('today')` and `pd.to_datetime('now')` are resolved in nanoseconds.

```
print((pd.to_datetime('today') - pd.to_datetime('YYYY-MM-DD')).days)
print((pd.to_datetime('now') - pd.to_datetime('YYYY-MM-DDTHH:MM')).total_seconds())
```

Exercise 2

```
print(pd.to_datetime('YYYY-MM-DD').day_of_week)
```

Exercise 3

```
(pd.to_datetime('2025-12-25') - pd.to_datetime('now')).days
```

Exercise 4

```
leap_years = pd.date_range(start = '1901', end = '2000', freq = 'YE')
leap_years = leap_years[leap_years.is_leap_year]
print(leap_years.year)
```

```
Index([1904, 1908, 1912, 1916, 1920, 1924, 1928, 1932, 1936, 1940, 1944, 1948,
       1952, 1956, 1960, 1964, 1968, 1972, 1976, 1980, 1984, 1988, 1992, 1996],
      dtype='int32')
```

6 Reading and Writing Files

Pandas provides a set of functions to read and write files that follow a consistent naming convention. Functions for reading files are called as `pd.read_csv()`, and functions for writing files are called as `pd.to_csv()`. Pandas can also read files from the internet using `pd.read_csv(URL)`.

Format	Type	Data Description	Reader	Writer
	text	CSV	<code>read_csv</code>	<code>to_csv</code>
	text	Fixed-Width Text File	<code>read_fwf</code>	NA
	text	JSON	<code>read_json</code>	<code>to_json</code>
	text	HTML	<code>read_html</code>	<code>to_html</code>
	text	LaTeX	<code>Styler.to_latex</code>	NA
	text	XML	<code>read_xml</code>	<code>to_xml</code>
	text	Local clipboard	<code>read_clipboard</code>	<code>to_clipboard</code>
binary		MS Excel	<code>read_excel</code>	<code>to_excel</code>
binary		OpenDocument	<code>read_excel</code>	NA
binary		HDF5 Format	<code>read_hdf</code>	<code>to_hdf</code>
binary		Feather Format	<code>read_feather</code>	<code>to_feather</code>
binary		Parquet Format	<code>read_parquet</code>	<code>to_parquet</code>
binary		ORC Format	<code>read_orc</code>	<code>to_orc</code>
binary		Stata	<code>read_stata</code>	<code>to_stata</code>
binary		SAS	<code>read_sas</code>	NA
binary		SPSS	<code>read_spss</code>	NA
binary		Python Pickle Format	<code>read_pickle</code>	<code>to_pickle</code>
SQL		SQL	<code>read_sql</code>	<code>to_sql</code>

([Pandas Documentation](#))

The following demonstrates reading the `palmerpenguins` dataset using Pandas.

palmerpenguins

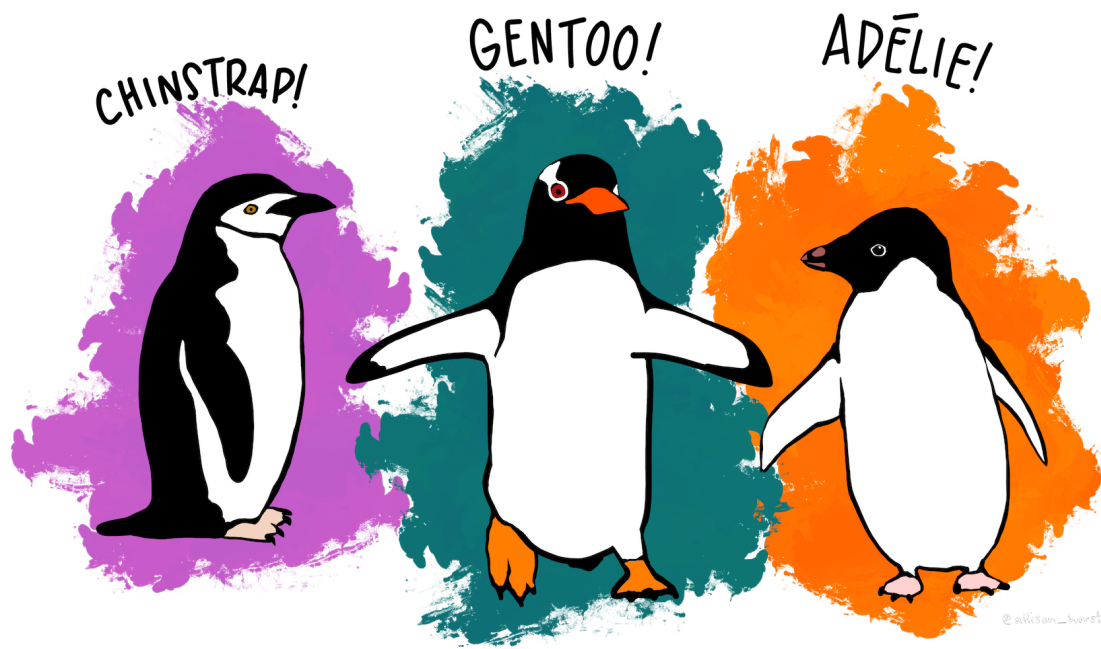


Figure 6.1: Penguins from the Palmer Station dataset

“Meet the Palmer Penguins” by @allison_horst is licensed under [CC0-1.0](#) and is available on [GitHub](#). 2020

The dataset is licensed under [CC0](#) and is available in R as well as on [GitHub](#). 2020

```
# R commands to load the dataset
install.packages("palmerpenguins")
library(palmerpenguins)
```

Horst AM, Hill AP, and Gorman KB. 2020. palmerpenguins: Palmer Archipelago (Antarctica) penguin data. R package version 0.1.0. <https://allisonhorst.github.io/palmerpenguins/>. doi: 10.5281/zenodo.3960218.

File-reading functions expect a path, which can be provided either positionally or as a keyword argument. The keyword depends on the file type; for a comma-separated CSV file, it is `filepath_or_buffer`.

```
penguins = pd.read_csv(filepath_or_buffer = '01-daten/penguins.csv')
```

A quick look at the data using the `penguins.head()` method:

```
print(penguins.head())
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	

	body_mass_g	sex	year
0	3750.0	male	2007
1	3800.0	female	2007
2	3250.0	female	2007
3	NaN	NaN	2007
4	3450.0	female	2007

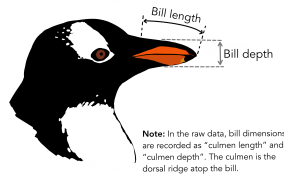


Figure 6.2: Bill Dimensions

“Bill dimensions” by @allison_horst is licensed under [CC0-1.0](#) and is available on [GitHub](#). 2020

An overview of the dataset can be obtained using the `DataFrame.info()` method.

```
print(penguins.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
0	species	344 non-null	str
1	island	344 non-null	str
2	bill_length_mm	342 non-null	float64
3	bill_depth_mm	342 non-null	float64
4	flipper_length_mm	342 non-null	float64
5	body_mass_g	342 non-null	float64
6	sex	333 non-null	str
7	year	344 non-null	int64

dtypes: float64(4), int64(1), str(3)

memory usage: 21.6 KB

None

Some data types were not automatically recognized. The affected columns were assigned the generic type `object`. You can specify the data type when reading a file using the `dtype` argument. For multiple columns, this can be done with a dictionary in the form `{'column_name': 'dtype'}`. Using the `DataFrame.astype()` method, this can also be done afterwards.

```
penguins = pd.read_csv(filepath_or_buffer = '01-daten/penguins.csv', dtype = {'species': 'category'})

# Afterwards
# penguins = penguins.astype({'species': 'category', 'island': 'category', 'sex': 'category'})

print(penguins.info())
```

<class 'pandas.DataFrame'>

RangeIndex: 344 entries, 0 to 343

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	species	344 non-null	category
1	island	344 non-null	category
2	bill_length_mm	342 non-null	float64
3	bill_depth_mm	342 non-null	float64
4	flipper_length_mm	342 non-null	float64
5	body_mass_g	342 non-null	float64
6	sex	333 non-null	category
7	year	344 non-null	int64

dtypes: category(3), float64(4), int64(1)

memory usage: 15.0 KB

None

Some columns contain invalid values. Penguins with incomplete data should be removed from the dataset.

- Use `DataFrame.apply(pd.isna)` to identify missing values.
- Use `DataFrame.any(axis=1)` to aggregate row-wise. `any` returns True if at least one element is True.
- Use `sum()` to count the number of rows with missing values.
- Use `np.where()` to get their index positions.
- Use `DataFrame.drop()` to remove the corresponding rows.

```
# Identify missing values
print(penguins.apply(pd.isna).head(), "\n")

# Aggregate row-wise
print(penguins.apply(pd.isna).any(axis = 1).head(), "\n")

# Count rows with missing values
print(f"{penguins.apply(pd.isna).any(axis = 1).sum()} penguins have incomplete values.\n")

# Get index positions
print(np.where(penguins.apply(pd.isna).any(axis = 1))[0])

# Remove rows
penguins.drop(np.where(penguins.apply(pd.isna).any(axis = 1))[0], inplace = True)
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	False	False	False	False	False	
1	False	False	False	False	False	
2	False	False	False	False	False	
3	False	False	True	True	True	
4	False	False	False	False	False	

	body_mass_g	sex	year
0	False	False	False
1	False	False	False
2	False	False	False
3	True	True	False
4	False	False	False

0	False
1	False
2	False

```
3     True
4     False
dtype: bool
```

11 penguins have incomplete values.

```
[ 3   8   9  10  11  47 178 218 256 268 271]
```

Check the cleaned dataset:

```
print(penguins.info())
```

```
<class 'pandas.DataFrame'>
Index: 333 entries, 0 to 343
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                333 non-null   category
1   island                  333 non-null   category
2   bill_length_mm         333 non-null   float64
3   bill_depth_mm          333 non-null   float64
4   flipper_length_mm      333 non-null   float64
5   body_mass_g            333 non-null   float64
6   sex                     333 non-null   category
7   year                    333 non-null   int64
dtypes: category(3), float64(4), int64(1)
memory usage: 17.0 KB
None
```

6.1 Reading Time Series

With Pandas, reading time series data is straightforward. String parsing allows any string to be interpreted as a datetime.

If the internal structure of a file is known, the necessary parameters can be passed directly when reading, for example with `pd.read_csv()`. The parameters `parse_dates` and `date_format` are used for this.

`parse_dates` specifies where the datetime information is located. Different arguments can be passed:

- `parse_dates = True` interprets the index as datetime.

- A list of integers or column labels interprets these columns individually as datetime, e.g., `parse_dates = [1, 2, 3]`.
- A list of lists combines the specified columns into a single datetime column, e.g., `parse_dates = [[1, 2, 3]]`. The column values are concatenated with a space and then parsed.

Pandas interprets strings according to [ISO 8601](#) as a representation of a date in the order year, month, day, hour, minute, second, millisecond in the format `YYYY-MM-DD 12:00:00.000`. The separator between date and time can be a space or the letter T. The data type and the smallest unit are stored in the `dtype` attribute.

Other formats can be specified with the `date_format` parameter. Using the [strftime documentation](#), a non-ISO8601 format can be passed.

Datetime information can also be declared afterwards using `pd.to_datetime(arg, format="...")`. The `arg` parameter specifies the column to convert. The `format` parameter works like `date_format` to specify a non-ISO8601 format.

Time series data for Microsoft stock is stored at '01-daten/Microsoft_Stock.csv'.

Microsoft Stock - Time Series Analysis by Vijay V Venkitesh is licensed under [CC0](#) and is available on [Kaggle](#). 2021

```
stock = pd.read_csv(filepath_or_buffer = '01-daten/Microsoft_Stock.csv')

print(stock.head(), "\n")
print(stock.info())
```

	Date	Open	High	Low	Close	Volume
0	4/1/2015 16:00:00	40.60	40.76	40.31	40.72	36865322
1	4/2/2015 16:00:00	40.66	40.74	40.12	40.29	37487476
2	4/6/2015 16:00:00	40.34	41.78	40.18	41.55	39223692
3	4/7/2015 16:00:00	41.61	41.91	41.31	41.53	28809375
4	4/8/2015 16:00:00	41.48	41.69	41.04	41.42	24753438

```
<class 'pandas.DataFrame'>
RangeIndex: 1511 entries, 0 to 1510
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        1511 non-null   str
1   Open        1511 non-null   float64
2   High        1511 non-null   float64
3   Low         1511 non-null   float64
```

```

4   Close    1511 non-null   float64
5   Volume   1511 non-null   int64
dtypes: float64(4), int64(1), str(1)
memory usage: 71.0 KB
None

```

The `Date` column contains date and time information in the format ‘Month/Day/Year Hour:Minute:Second’, which Pandas did not automatically recognize. Therefore, the column has the data type `object`.

6.2 Time Series Reading Exercises

1. Pass the required arguments to `pd.read_csv()` to correctly read the `Date` column as `datetime`.
2. Calculate the weekly high prices (intraday).

💡 Tip 12: Solution for Reading Time Series

1. Task

```

stock = pd.read_csv(filepath_or_buffer = '01-daten/Microsoft_Stock.csv',
                    parse_dates = ['Date'], # alternatively: [0]
                    date_format = '%m/%d/%Y %H:%M:%S')

print(stock.head(), "\n")
print(stock.info())

```

	Date	Open	High	Low	Close	Volume
0	2015-04-01 16:00:00	40.60	40.76	40.31	40.72	36865322
1	2015-04-02 16:00:00	40.66	40.74	40.12	40.29	37487476
2	2015-04-06 16:00:00	40.34	41.78	40.18	41.55	39223692
3	2015-04-07 16:00:00	41.61	41.91	41.31	41.53	28809375
4	2015-04-08 16:00:00	41.48	41.69	41.04	41.42	24753438

```

<class 'pandas.DataFrame'>
RangeIndex: 1511 entries, 0 to 1510
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        1511 non-null   datetime64[us]

```

```

1   Open      1511 non-null   float64
2   High      1511 non-null   float64
3   Low       1511 non-null   float64
4   Close     1511 non-null   float64
5   Volume    1511 non-null   int64
dtypes: datetime64[us](1), float64(4), int64(1)
memory usage: 71.0 KB
None

```

2. Task

The Pandas method `Series.dt.weekofyear()` is no longer supported ([see documentation](#)). It has been replaced by `Series.dt.isocalendar().week`.

```

# Extract year and week
print(stock['Date'].dt.isocalendar().week.head(), "\n")
print(stock['Date'].dt.isocalendar().year.tail())

# Insert year and week into the DataFrame
stock.insert(loc = 1, column = 'week', value = stock['Date'].dt.isocalendar().week)
stock.insert(loc = 1, column = 'year', value = stock['Date'].dt.isocalendar().year)

# Determine weekly maximum using groupby
print(stock.groupby(by = ['year', 'week'])['High'].max())

# Plot the weekly high prices
stock.groupby(by = ['year', 'week'])['High'].max().plot(
    ylabel = 'Weekly High Price (intraday)',
    title = 'Microsoft Stock Price Development'
)

```

```

0      14
1      14
2      15
3      15
4      15
Name: week, dtype: UInt32

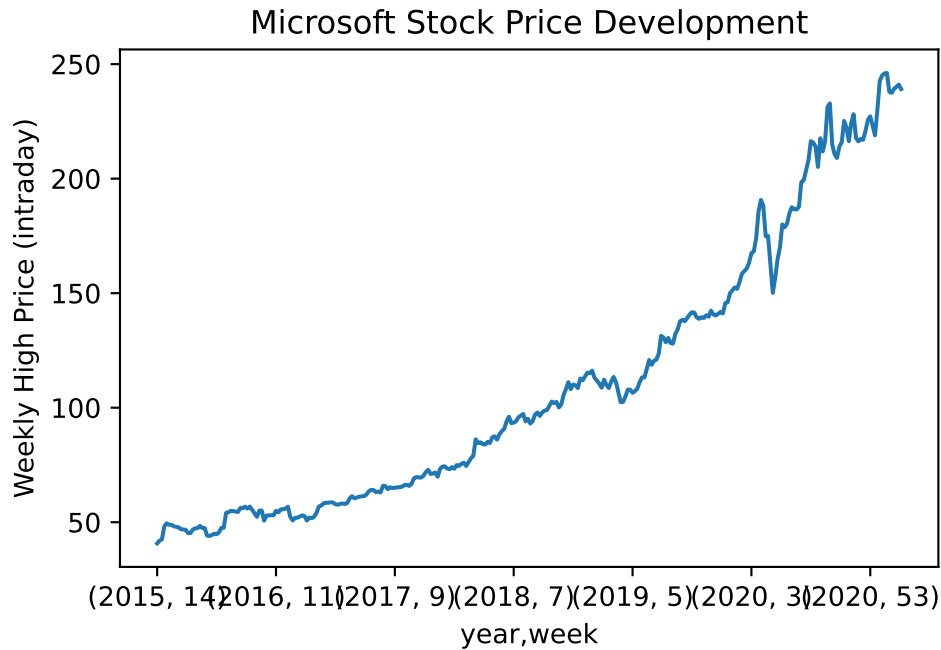
1506    2021
1507    2021
1508    2021
1509    2021

```

```

1510      2021
Name: year, dtype: UInt32
year  week
2015   14      40.76
      15      41.95
      16      42.46
      17      48.14
      18      49.54
      ...
2021    9      237.47
      10      239.17
      11      240.06
      12      241.05
      13      239.10
Name: High, Length: 314, dtype: float64

```



6.3 Reading Complex Files

Reading files is not always straightforward. Tools and strategies for handling difficult cases can be found in the [Structured Dataset Reading Guide](#). This guide also provides a detailed discussion on handling missing values.